

Programmation WEB

PR. SOUKAINA MJAHER

s.mjahed@uca.ac.ma

Plan du cours

❑ Partie 1 : Introduction au Web

- Web et internet : Présentation et historique
- Architecture Client/Serveur
- Protocole HTTP
- Web statique et Web dynamique

❑ Partie 2 : HTML5

- Présentation du langage HTML et notions de base
- Balises de formatage du texte
- Paragraphes, conteneurs et listes
- Images et liens hypertextes
- Les tableaux HTML
- Les formulaires
- Les balises sémantiques : Nav, Head, Section ...

❑ Partie 3 : Feuilles de styles (CSS)

- Introduction aux feuilles de style
- Définition des règles css
- Syntaxe des règles css (Selecteur, class et ID)
- Cascade, conflits et héritages
- Les unités de longueur
- Propriétés CSS pour le texte, les listes et les tableaux
- Les boîtes : définition, remplissage, marges et positionnement (Flottement)
- Les éléments de type : block, inline, non affichés
- Les flexbox
- Pseudo classes de liens
- CSS pour les tableaux
- Responsive Design : media et queries
- Introduction au framework Bootstrap

Plan du cours

❑ **Partie 4 : Langage Javascript**

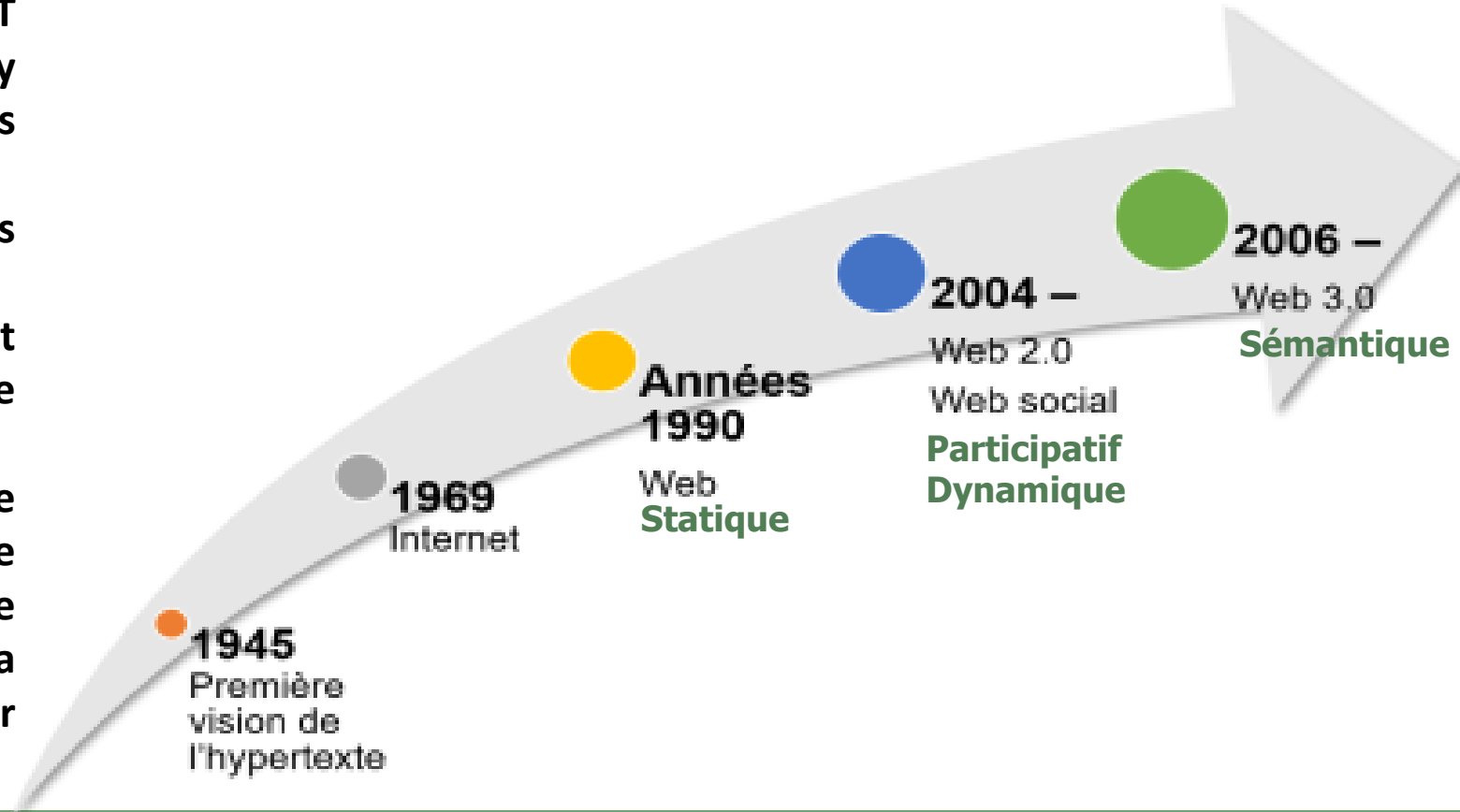
- **Introduction au langage JavaScript**
- **Notion de variable, constante et type**
- **Les boîtes de dialogue**
- **Les structures conditionnelles et répétitives**
- **Les fonctions et les fonctions anonymes**
- **Les tableaux**
- **Le Document Object Model (DOM) : Présentation, identification, récupération et ajout des éléments**
- **Evénements et EventListener**
- **JavaScript et formulaire**
- **Introduction au framework AngularJs**

Partie 1 : Introduction au Web

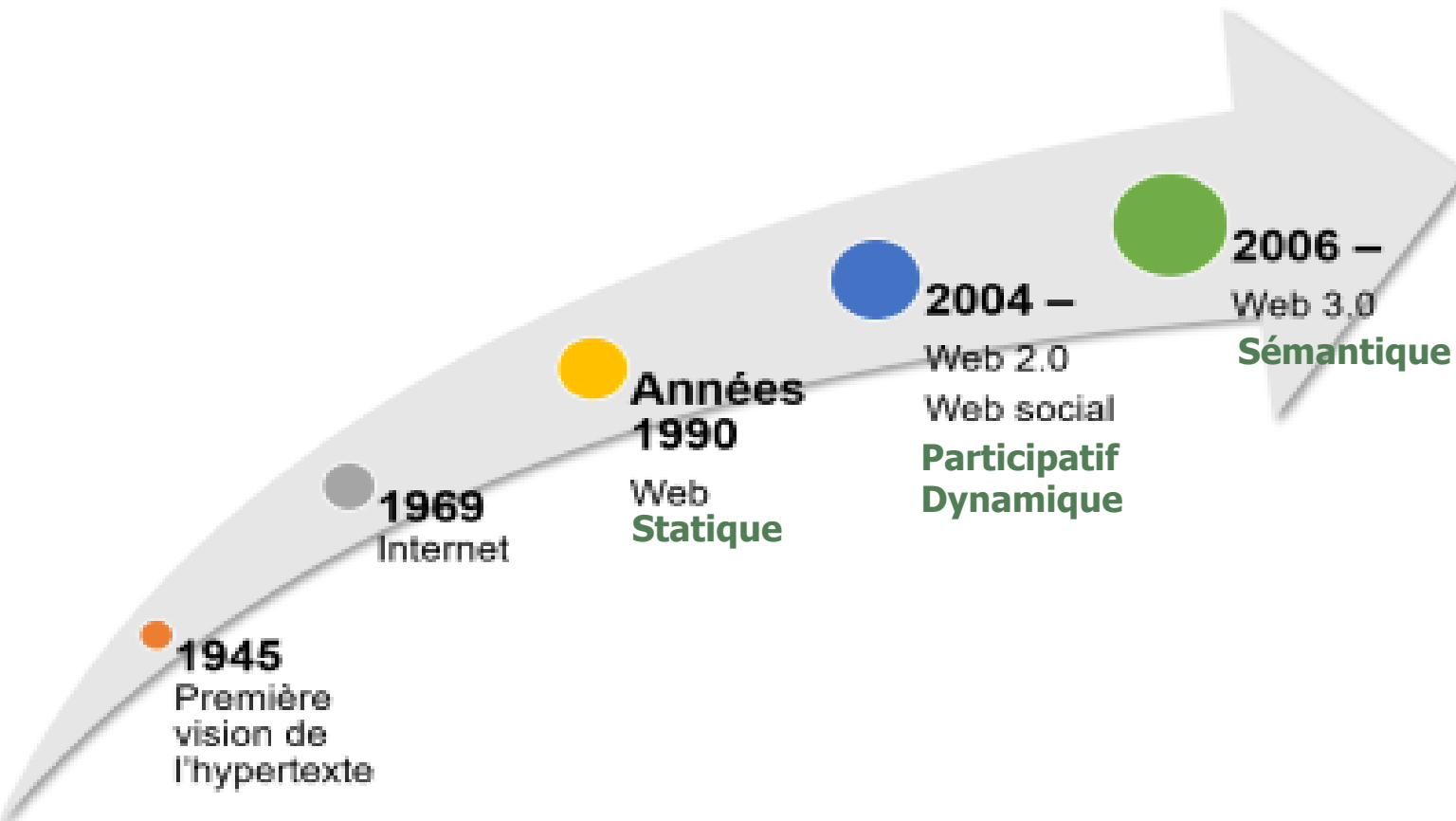
Introduction au Web : « Web et internet : Présentation et historique »

✓ Internet :

- Internet est né d'une initiative militaire américaine.
- Le premier nœud du réseau ARPANET (Advanced Research Project Agency Network) à l'origine d'Internet a été mis en place en 1969.
- Avant l'apparition du Web, les échanges sur Internet étaient uniquement textuels.
- Internet est un réseau large regroupant différentes sphères (Web, courriel, File Transfer Protocol (FTP), etc.).
- Lors de la consultation d'une page internet, on n'a accès qu'à une petite partie des fichiers installés sur la machine serveur. Ceci dépend de la limitation de la zone accessible depuis l'extérieur (nécessaire pour des raisons de sécurité).



Introduction au Web : « Web et internet : Présentation et historique »



✓ Web :

- Un ensemble cohérent de pages Web liées par des liens hypertextes et articulées autour d'une page d'accueil commune est appelée site web.
- World Wide Web (noté WWW) ou tout simplement Web (mot anglais signifiant toile) la "toile virtuelle".
- Le Web est une des sphères d'Internet.
- W3C : (World Wide Web Consortium) est un organisme de standardisation à but non lucratif. Il se charge de fixer les standards du Web comme les langages HTML ou CSS, ou encore les protocoles utilisés sur la toile. Tout comme le WWW, il a été fondé par Tim Berners-Lee en 1994. C'est d'ailleurs lui qui en est le Directeur.

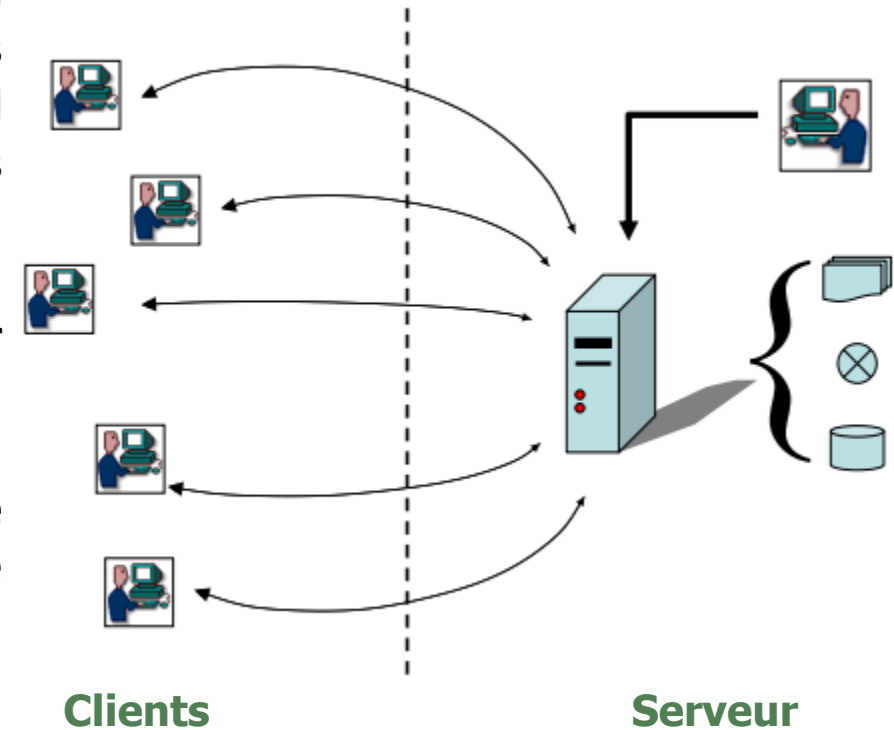
Introduction au Web : « Web statique et Web dynamique »

- ✓ **Web statique** : est simple, rapide et adapté aux sites qui n'ont pas besoin de beaucoup d'interactions ou de changements fréquents.
- ✓ **Web dynamique** : est plus puissant, flexible, et indispensable pour des applications web interactives nécessitant des mises à jour fréquentes ou des interactions avec des bases de données.

Web statique	Web dynamique
Contenu fixe	Contenu interactif et personnalisé
HTML/CSS/Javascript	PHP, Python, etc., avec base de données
Rapide et simple	Plus complexe et demande plus de ressources
Aucun changement en fonction de l'utilisateur	Contenu généré instantanément
Sites vitrines, portfolios, blogs simples	E-commerce, réseaux sociaux, applications web, systèmes de gestion de contenu (CMS)

Introduction au Web : « Architecture Client/Serveur »

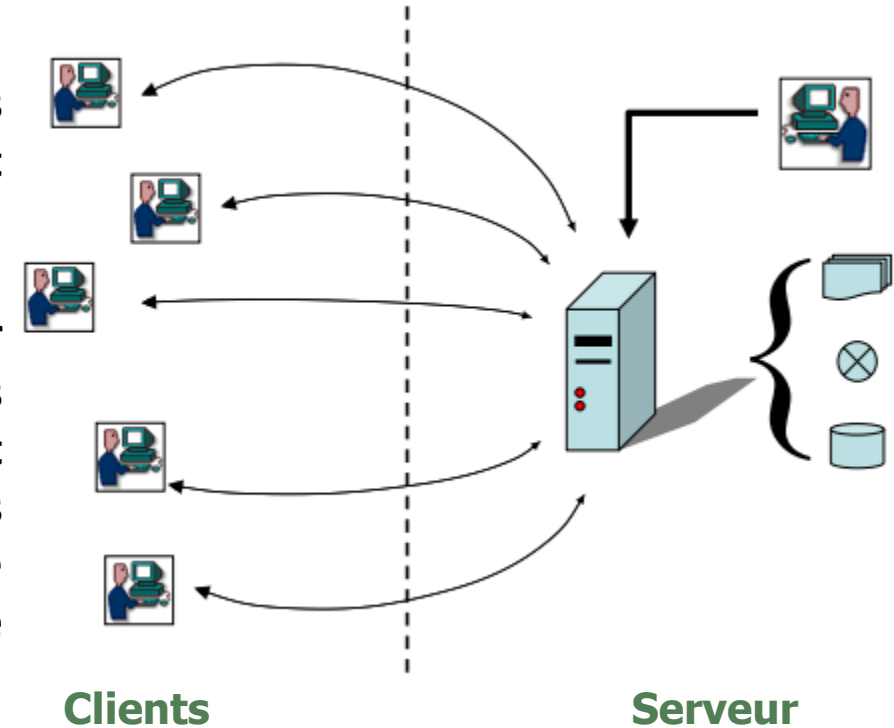
- ✓ L'architecture client-serveur : Lorsqu'on connecte les ordinateurs en réseau, il devient intéressant de concentrer certaines ressources (fichiers, applications, bases de données, programmes) sur un seul ordinateur, et de permettre aux autres ordinateurs de se servir de ces ressources uniquement lorsqu'ils en ont besoin.
- ✓ Un client et un serveur peuvent être très éloignés l'un de l'autre, par exemple lorsqu'on utilise **Internet**.
- ✓ Le serveur joue en premier lieu un rôle de distributeur de fichiers (suite à des **requêtes**). Autour de ce rôle de serveur de fichiers, il peut rendre diverses sortes de services.
- ✓ A côté des clients, le serveur possède un **administrateur**, qui l'alimente en données, en programme et en base de données.
- ✓ NB : Lorsqu'on parle de « client » et de « serveur », il peut s'agir à la fois de l'ordinateur et du programme qui tourne sur cet ordinateur.



Introduction au Web : « Architecture Client/Serveur »

✓ Avantages :

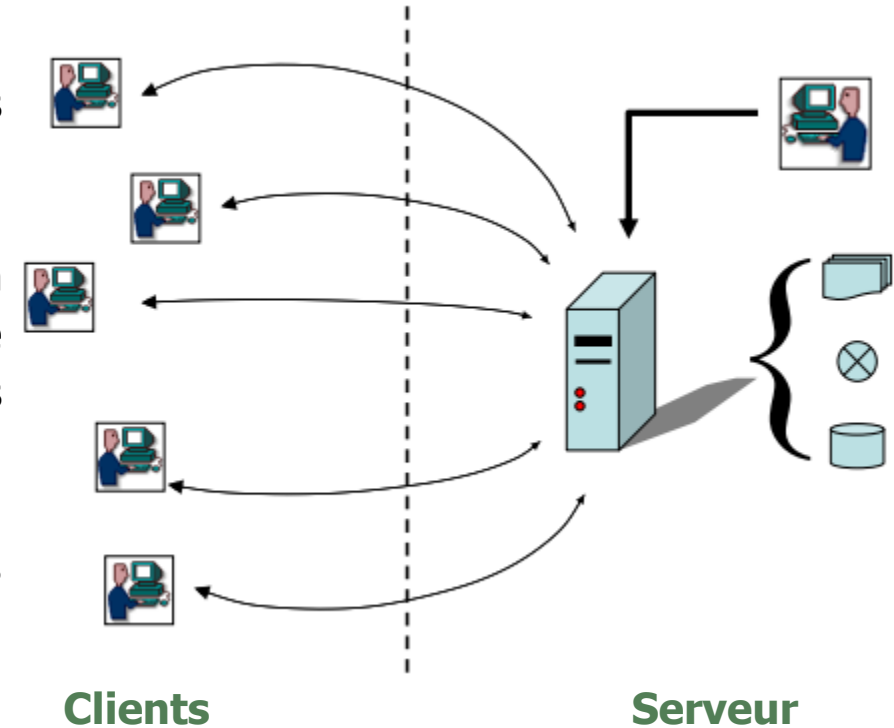
- une économie d'espace mémoire (tous les postes clients vont chercher l'information en fonction de leurs besoins, et ne doivent pas la stocker durablement)
- une garantie de cohérence (à chaque instant, le serveur constitue un référentiel unique, identique pour tous les clients, alors qu'une duplication sur chaque poste client entraîne généralement une divergence entre les différentes versions, et une perte du référentiel commun). Cette garantie de cohérence est particulièrement souhaitable lorsque l'information subit des mises à jour.
- Une maîtrise du service (le gestionnaire du serveur contrôle ce que les programmes clients peuvent faire ou ne pas faire sur le serveur).



Introduction au Web : « Architecture Client/Serveur »

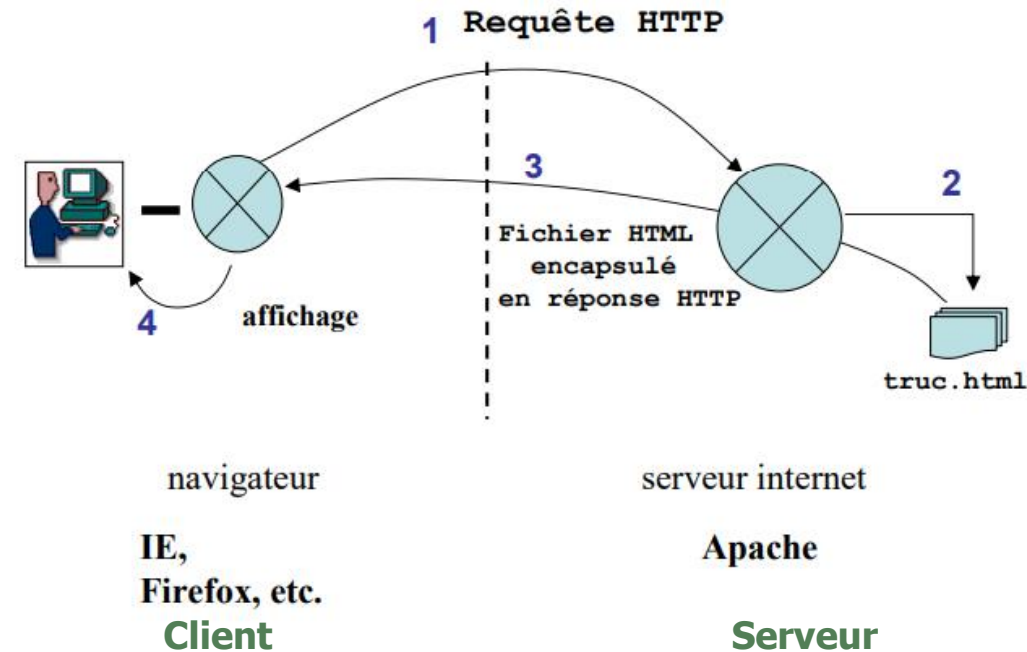
✓ Internet a une architecture client-serveur :

- Chaque serveur est potentiellement accessible par des millions de clients.
- Chacun est libre de décider de ce qu'il met en ligne, chacun possède en permanence le droit d'accéder à un volume de données plus d'un million de fois supérieur à ses capacités de stockage personnelles.
- Sur le web, le serveur s'appelle « serveur internet ». Exemple : Apache.
- Sur le web, le client s'appelle navigateur, (ou bien browser, ou butineur). Exemple : Internet Explorer, Firefox, Opera, Chrome.



Introduction au Web : « Protocole HTTP »

- ✓ Un protocole : est une convention de communication entre deux ordinateurs.
- ✓ Les protocoles d'Internet :
 - TCP/IP : correspond à deux protocoles imbriqués « TCP » et « IP ». C'est le protocole de base qui structure Internet (le web et les autres composants d'Internet, comme le FTP et le mail). TCP/IP permet de transformer les signaux physiques en information numérique. Dans la programmation d'un site web, on utilise un protocole logique qui s'appuie sur TCP/IP.
 - HTTP : (Hyper-Text Transfert Protocole) est le protocole spécifique du web. C'est donc le langage avec lequel le serveur et le client dialoguent.

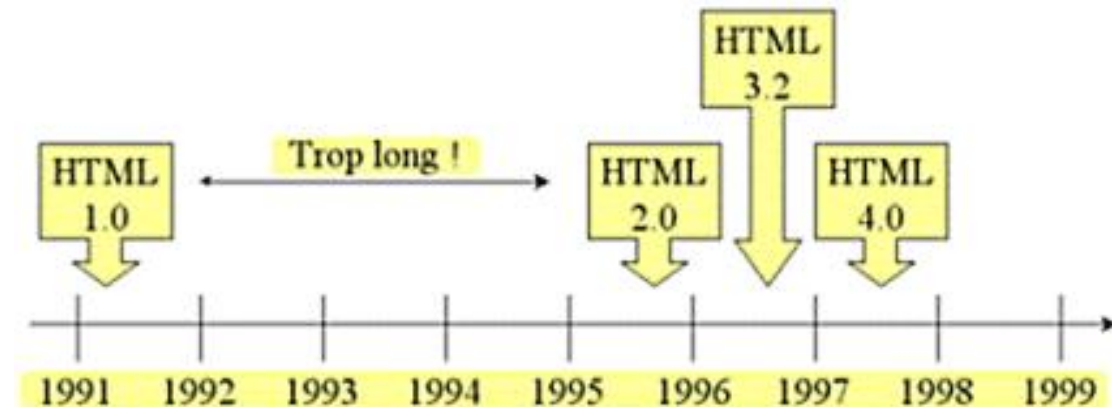


- 1- le navigateur demande un fichier au serveur,
- 2- le serveur va chercher le fichier dans son système de fichier,
- 3- le serveur poste le fichier au navigateur,
- 4- le navigateur reçoit le fichier et l'affiche.

Partie 2 : HTML 5

HTML 5 : « Présentation du langage HTML et notions de base »

- ✓ **HTML** : (Hyper Text Mark-up Language) est un langage de description (dit de marquage) de pages Web. Il permet de présenter les documents hypertextes destinés à être affichés sur le navigateur. Il s'agit d'un langage coté client (tout comme CSS et Javascript). Il est supporté et développé par W3C.
- ✓ **Format d'une page HTML** :
 - Une page HTML est un fichier texte avec l'extension .html ou .htm (aucune différence entre les deux extensions).
 - Les fichiers HTML sont déposés dans un serveur de fichiers. Il ne nécessite donc aucune exécution par le serveur car HTML est un langage coté client. Par conséquent, il n'y a aucun problème à le mettre un fichier HTML n'importe où dans votre ordinateur car c'est le navigateur qui l'exécutera après ouverture.
 - Par convention, la page d'accueil d'un site Web est nommée "index.html".



HTML 5 : « Présentation du langage HTML et notions de base »

✓ Exemple d'une page HTML :

```
<html>
  <head>
    <title>Ma première page HTML</title>
    <meta http-equiv="content-type"
content="text/html; charset=utf-8">
  </head>
  <body>
    <h1>Il s'agit d'un exemple de HTML</h1>
    <div>
      <font color="#DD7700"><b>HTML</b></font>
      est un langage de description de pages Web.
    </div>
  </body>
</html>
```

HTML 5 : « Présentation du langage HTML et notions de base »

✓ Notions de bases :

- **Balise** : élément de base du langage HTML qui a une signification que le navigateur connaît et applique au contenu. Elle est représentée par le chevron ouvrant, suivi par le nom de la balise puis du chevron fermant. Exemple `<b>` ou `<body>`.
- Il existe deux types de balises: des balises ouvrantes et des balises fermantes.
La balise ouvrante applique un effet au contenu qui la suit et la balise fermante cesse l'effet en question.
La balise fermante se distingue par un slash (/) qui vient juste avant le nom de la balise.
Exemple `</b>` est la balise fermante de `<b>` et `</body>` est la balise fermante de `<body>`.
- Il existe des balises ouvrantes qui n'ont pas de balises fermantes associées. On les appelle des balises auto fermantes ou des balises orphelines. Ce sont des balises qui s'ouvrent et se ferment au même moment et elles appliquent leur effet à l'endroit où elles sont déclarées.
Exemple: `<br>`.

HTML 5 : « Présentation du langage HTML et notions de base »

✓ Code HTML minimal :

```
<html>
  <head>
    <title>Titre de ma page Web</title>
  </head>
  <body>

  </body>
</html>
```

HTML 5 : « Présentation du langage HTML et notions de base »

✓ Principales balises HTML :

STRUCTURE DU DOCUMENT

<!DOCTYPE>	Version du (X)HTML
<html>	Document HTML
<head>	Informations de la page
<body>	Contenu de la page

INFORMATIONS DE LA PAGE

<title>	Titre de la page
<meta />	Méta-données
<base />	Adresse de la base
<link />	Ressource externe
<style>	Définition de styles
<script>	Définition de scripts

BALISES DE STRUCTURE

<div>	Enveloppe en bloc
	Enveloppe en chaîne
<section>	Section
<nav>	Navigation
<header>	Header
<footer>	Footer
<article>	Article
<aside>	Contenu associé / secondaire

LIENS

	Lien vers un fichier
	Lien vers un mail
	Ancre
	Lien vers une ancre

HTML 5 : « Présentation du langage HTML et notions de base »

✓ Principales balises HTML :

MISE EN FORME DU TEXTE

<h[1-6]>	Niveaux de titres
<p>	Paragraphe
 	Saut de ligne
	Mise en gras (SEO)
	Mise en gras
	Mise en italique (SEO)
<i>	Mise en italique
<blockquote>	Citation externe
<mark>	Surlignage
<pre>	Texte pré-formaté
<code>	Code
<address>	Adresse
<cite>	Citation

TABLEAUX

<table>	Tableau
<caption>	Titre du tableau
<thead>	En-tête du tableau
<tfoot>	Pied du tableau
<tbody>	Corps du tableau
<th>	Cellule d'en-tête
<tr>	Ligne
<td>	Cellule

LISTES

	Liste
	Liste ordonnée
	Élément d'une liste
<dl>	Liste de définitions
<dt>	Terme
<dd>	Définition

HTML 5 : « Présentation du langage HTML et notions de base »

✓ Principales balises HTML :

FORMULAIRES

<form>	Zone d'un formulaire
<input />	Champ d'un formulaire
<textarea>	Champ texte multi-lignes
<button>	Bouton
<select>	Liste déroulante
<optgroup>	Groupe d'options
<option>	Option d'une liste
<label>	Titre d'un élément
<fieldset>	Groupe de champs
<legend>	Titre d'un fieldset
<datalist>	Liste de choix
<output>	Résultat d'un calcul

MÉDIAS

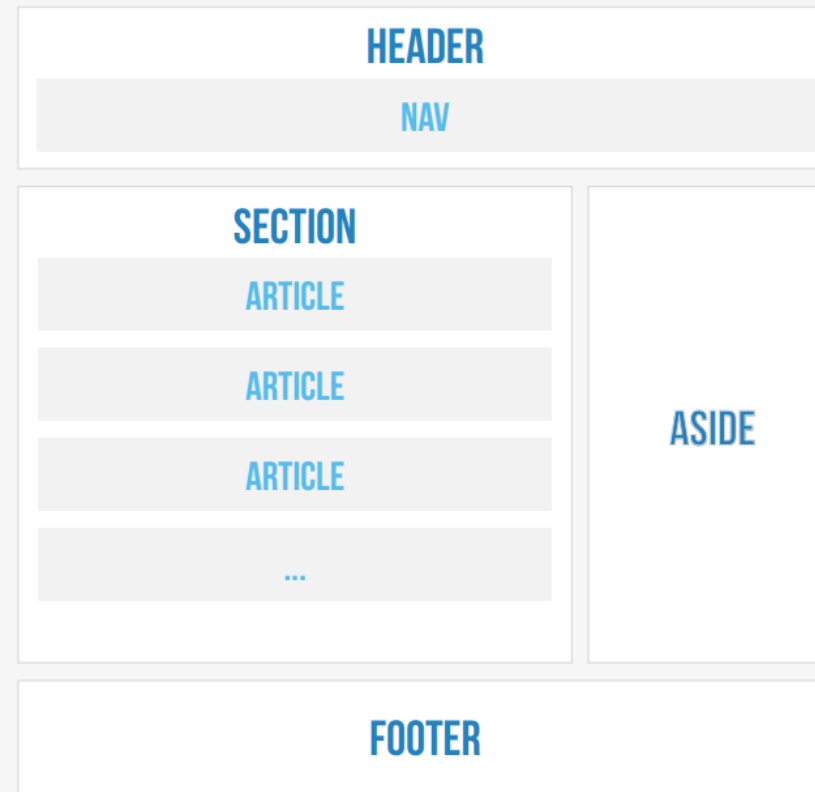
	Image
<map>	Zoning sur une image
<area>	Zone du zoning
<canvas>	Bloc graphique
<figure>	Conteneur de média
<figcaption>	Titre d'un conteneur
<audio>	Son
<source>	Ressources d'un média
<track>	Pistes d'un média
<video>	Vidéo
<object>	Object externe
<param>	Paramètre d'un objet

HTML 5 : « Présentation du langage HTML et notions de base »

✓ Principales balises HTML5 :

STRUCTURE D'UN DOCUMENT HTML 5

```
<html>
  <head>
    <title>Titre de la page</title>
    <meta charset="UTF-8" />
  </head>
  <body>
    <header>
      <nav></nav>
    </header>
    <section>
      <article></article>
      <article></article>
      <article></article>
      ...
    </section>
    <aside></aside>
    <footer></footer>
  </body>
</html>
```



HTML 5 : « Présentation du langage HTML et notions de base »

✓ Imbrication des balises :

- En cas d'imbrication de plusieurs balises, la première balise ouverte est la dernière à fermer. Il faut donc fermer les balises dans l'ordre inverse de leur ouverture.

- Syntaxe :

`<balise1><balise2><balise3>Contenu</balise3></balise2></balise1>`

- Exemple :

`<p>`

Ceci est un `<strong>`paragraphe`</strong>` avec des mots `<em>`mis en italique`</em>` et des mots en `<strong><em>`gras et italique`</em></strong>`.

`</p>`

HTML 5 : « Présentation du langage HTML et notions de base »

✓ Les attributs :

- Les attributs représentent des paramètres qui personnalisent la balise où ils sont définis et ils possèdent des valeurs. Ils sont déclarés dans la balise ouvrante. Une balise peut renfermer plusieurs attributs à la fois, chacun avec sa valeur.

- Syntaxe :

<balise attribut1="valeur1" attribut2="valeur2" attribut3="valeur3"...>Contenu</balise>

- Exemple :

Texte à afficher

Les options de l'attribut "target" sont :

- ❖ **_blank :** Ouverture de la page cible dans une nouvelle fenêtre
- ❖ **_parent :** Ouverture de la page cible dans le cadre parent
- ❖ **_self :** Ouverture de la page cible dans le cadre hôte
- ❖ **_top :** Ouverture de la page cible dans la fenêtre hôte

HTML 5 : « Présentation du langage HTML et notions de base »

✓ Les principaux caractères spéciaux :

HTML ne prend pas en compte la succession d'espaces blancs, seul le premier est considéré. Pour forcer plusieurs espaces successifs il faut passer par le code HTML en utilisant plusieurs ** **

Caractère	Code HTML
"	"
&	&
<	<
>	>
œ	œ
Espace	
£	£
©	©
®	®
±	±
μ	µ
½	½
Ç	Ç
æ	æ

HTML 5 : « Présentation du langage HTML et notions de base »

✓ Balises de formatage de texte sans attributs :

- Balise et :

La balise **** met le texte en gras (bold). C'est utile pour les textes importants ou les titres.

- Exemple :

```
<b>Ce texte est écrit en gras</b>
```

Ce texte est écrit en gras

- La balise **** met aussi le texte en gras, tout comme la balise ****.

HTML 5 : « Présentation du langage HTML et notions de base »

✓ Balises de formatage de texte sans attributs :

▪ Balise <u> :

La balise **<u>** souligne le texte qu'elle entoure. C'est utile pour attirer l'attention à un mot ou une phrase.

▪ Exemple :

```
<u>Texte souligné</u>
```

Texte souligné

HTML 5 : « Présentation du langage HTML et notions de base »

✓ Balises de formatage de texte sans attributs :

- Balise <i> :

La balise **<i>** met le texte en italique (pour les citations ou les notes).

- Exemple :

```
<i>Texte en italique</i>
```

Texte en italique

HTML 5 : « Présentation du langage HTML et notions de base »

✓ Balises de formatage de texte sans attributs :

▪ Balise <s> :

La balise **<s>** barre le texte. Elle sert pour marquer un contenu obsolète ou invalide (par exemple barrer les anciens prix d'articles en promotion sur les sites E-commerce).

▪ Exemple :

```
<s>Texte barré</s>
```

~~Texte barré~~

HTML 5 : « Présentation du langage HTML et notions de base »

✓ Balises de formatage de texte sans attributs :

▪ Balises <h1>, <h2>...<h6> :

La balise **<h1>** désigne un titre de premier niveau. Le texte qu'elle entoure est de grande taille et mis en gras. Il est utile pour créer les grands titres d'une page.

La balise **<h2>** désigne un titre de deuxième niveau. Elle applique le même effet que la balise précédente mais avec une taille de caractères légèrement plus petite. Elle est utilisée pour les sous-titres.

Il existe aussi **<h3>**, **<h4>**, **<h5>** et **<h6>** avec une taille de caractères de plus en plus petite.

▪ Exemple :

```
<h1>Titre de premier niveau</h1>
<h2>Titre de deuxième niveau</h2>
<h3>Titre de troisième niveau</h3>
<h4>Titre de quatrième niveau</h4>
<h5>Titre de cinquième niveau</h5>
<h6>Titre de sixième niveau</h6>
```

Titre de premier niveau

Titre de deuxième niveau

Titre de troisième niveau

Titre de quatrième niveau

Titre de cinquième niveau

Titre de sixième niveau

HTML 5 : « Présentation du langage HTML et notions de base »

✓ Balises de formatage de texte sans attributs :

▪ Balises <sup> et <sub> :

La balise **<sup>** met un texte en exposant et la balise **<sub>** le met en indice

▪ Exemple :

```
x<sup>2</sup>
```

x^2

```
H<sub>2</sub>O
```

H_2O

HTML 5 : « Présentation du langage HTML et notions de base »

✓ Balises de formatage de texte sans attributs :

▪ Balise
 :

Pour retourner à la ligne à n'importe quel moment, il suffit de déclarer la balise **
**.

Il s'agit d'une balise orpheline (donc pas de fermeture).

▪ Exemple :

```
Ligne 1 <br>  
Ligne 2 <br>  
Et ligne 3
```

```
Ligne 1  
Ligne 2  
Et ligne 3
```

HTML 5 : « Présentation du langage HTML et notions de base »

✓ Balises de formatage de texte sans attributs :

▪ Commentaire HTML <!-- --> :

Un commentaire HTML est déclaré comme ceci: **<!-- Commentaire -->**. Il est visible dans le code source mais ignoré par le navigateur. Il sert à marquer un bloc de code pour que celui-ci soit facilement trouvé et compris lors de sa prochaine manipulation.

▪ Exemple :

```
<!-- Ceci est un commentaire -->  
<i>Le commentaire n'est pas visible sur le navigateur.</i>
```

Le commentaire n'est pas visible sur le navigateur.

HTML 5 : « Présentation du langage HTML et notions de base »

✓ Balises de formatage de texte sans attributs :

▪ Balise <pre> :

Permet d'afficher un contenu tel qu'il a été édité dans le code. C'est à dire, si une tabulation dans est mise sur le code, celle-ci sera automatiquement reprise dans le navigateur, et si un saut de ligne est prévu, ce saut de ligne sera aussi visible sur le navigateur.

▪ Exemple :

```
<pre>
  Là-haut sur la montagne, l'était un vieux chalet.
  Murs blancs, toit de bardeaux,
  Devant la porte un vieux bouleau.
  Là-haut sur la montagne, l'était un vieux chalet.
</pre>
```

Là-haut sur la montagne, l'était un vieux chalet.
Murs blancs, toit de bardeaux,
Devant la porte un vieux bouleau.
Là-haut sur la montagne, l'était un vieux chalet.

HTML 5 : « Présentation du langage HTML et notions de base »

✓ Balises de formatage de texte avec attributs :

▪ Balise :

- Obsolète par W3C mais les navigateurs continuent à la prendre en charge.
- La balise doit sa notoriété aux effets qu'elle peut apporter aux textes qu'elle inclue.
- Elle dispose de trois attributs principaux qui permettent d'appliquer des effets aux textes. Ces trois attributs sont: face, size, et color.
- Attribut « face » :
 - L'attribut « face » permet de spécifier la police à utiliser pour afficher le texte inclus dans la balise .
 - Par défaut c'est la police "Times New Roman" qui est appliquée par la plupart des navigateurs si aucune police n'est spécifiée.
 - Pour que la police soit exécutée correctement, il faut qu'elle soit disponible sur le système d'exploitation du client.

• Exemple :

```
<font face="Arial">Ceci est un texte en Arial.</font>
```

Ceci est un texte en Arial.

HTML 5 : « Présentation du langage HTML et notions de base »

✓ Balises de formatage de texte avec attributs :

▪ Balise :

• Attribut « size » :

- L'attribut size change la police du texte.
- Par défaut le navigateur applique la taille 12 points (écrit 12pt) aux polices.
- HTML ne supporte que 7 tailles de police, elles sont déclarées par des indices allant de 1 à 7.
- Limitation des possibilités de redimensionner un texte aisément. HTML sert essentiellement à la création du contenu. Le design personnalisable d'une page est à traiter en utilisant le CSS (Cascading Style Sheets).

• Exemple :

Indice (valeur de l'attribut size)	Valeur réelle en pt
1	8pt
2	10pt
3 (par défaut)	12pt
4	14pt
5	18pt
6	24pt
7	36pt

```
<font size="6">Ceci est un texte avec une taille de 24pt.</font>
```

Ceci est un texte avec une taille de 24pt.

HTML 5 : « Présentation du langage HTML et notions de base »

✓ Balises de formatage de texte avec attributs :

▪ Balise :

• Attribut « color » :

- Sert à appliquer une couleur au texte entouré par la balise .
- La couleur peut être définie soit par son nom anglais (red pour rouge, yellow pour jaune...) qui correspond à environ 100 couleurs ou par son code qui correspond à 16 millions de couleurs.
- Pour qu'un écran affiche une image en couleur, il combine 3 couleurs dites primaires. Ces couleurs sont Rouge, Vert et Bleu. On parle alors de la base RVB (RGB en anglais).
- Chaque couleur primaire est codée sur un octet (8 bits). Les trois couleurs sont donc codées sur 24 bits et du coup on peut avoir jusqu'à 2^{24} nuances différentes (ce qui donne 16777216 couleurs).

HTML 5 : « Présentation du langage HTML et notions de base »

✓ Balises de formatage de texte avec attributs :

▪ Balise :

• Attribut « color » :

- En binaire :
 - ✓ Rouge: 11111111
 - ✓ Vert: 00000000
 - ✓ Bleu: 00000000
- Le système hexadécimal regroupe les bits 4 par 4.
- Exemple :

```
<font color="#FF0000" face="verdana" size="2">  
  Ce texte a été formaté grâce à la balise font  
</font>
```

Ce texte a été formaté grâce à la balise font

couleur	Code HTML
Rouge	#FF0000
Vert	#00FF00
Bleu	#0000FF
Blanc	#FFFFFF
Noir	#000000
Jaune	#FFFF00
Magenta	#FF00FF
Cyan	#00FFFF
Gris moyen	#888888
Orange	#FF8800

HTML 5 : « Présentation du langage HTML et notions de base »

✓ Balises de formatage de texte avec attributs :

▪ Balise <p> :

- La balise <p> crée un bloc et engendre automatiquement un retour à la ligne. Elle sert à définir un paragraphe.
- Attribut « align » :
 - Permet d'aligner le contenu du paragraphe.
 - Les différentes valeurs de l'attribut align sont:
 - left: C'est la valeur par défaut. Permet d'aligner le contenu du paragraphe à gauche du conteneur qui renferme la balise <p>.
 - right: Permet d'aligner le contenu du paragraphe à droite du conteneur qui renferme la balise.
 - center: centrer le paragraphe.
 - justify: Permet de justifier le contenu du paragraphe.
- Exemple :

HTML 5 : « Présentation du langage HTML et notions de base »

✓ Balises de formatage de texte avec attributs :

▪ Balise <p> :

- La balise <p> crée un bloc et engendre automatiquement un retour à la ligne. Elle sert à définir un paragraphe.
- Attribut « align » :
 - Exemple :

```
<h1>Les paragraphes</h1>
```

```
<p align="left">
```

```
    La balise &lt;p&gt; permet de définir un paragraphe au sein  
d'un code HTML.
```

```
    Il engendre un retour à la ligne avant et après.
```

```
</p>
```

Les paragraphes

La balise <p> permet de définir un paragraphe au sein d'un code HTML. Il engendre un retour à la ligne avant et après.

HTML 5 : « Présentation du langage HTML et notions de base »

✓ Les listes :

- Balise : liste ordonnée (ou numérotée)
 - Contient une autre balise qui est .
 - La balise permet de définir une entrée de la liste (Il y 'a donc autant de balises que d'entrées).
 - Attribut « type » :
 - Sert à définir le type de marqueur de la liste :
 - « 1 » pour les chiffres classiques (valeur par défaut).
 - « A » pour l'alphabet majuscule.
 - « a » pour l'alphabet minuscule.
 - « I » pour les chiffres romains majuscules
 - « i » pour les chiffres romains minuscule.

HTML 5 : « Présentation du langage HTML et notions de base »

✓ Les listes :

- Balise : liste ordonnée (ou numérotée)
 - Attribut « start » :
 - Indique la valeur du marqueur du début. Par défaut la liste commence par 1.
 - Exemple :

Les noms des balises de bases à déclarer dans un document HTML sont:

```
<ol type="1">  
  <li>HTML</li>  
  <li>HEAD</li>  
  <li>BODY</li>  
</ol>
```

Les noms des balises de bases à déclarer dans un document HTML sont:

1. HTML
2. HEAD
3. BODY

HTML 5 : « Présentation du langage HTML et notions de base »

✓ Les listes :

▪ Balise : liste non ordonnée (ou numérotée)

- Contient une autre balise qui est .
- La balise permet de définir une entrée de la liste (Il y 'a donc autant de balises que d'entrées).
- Attribut « type » :
 - **disc** : le marqueur est un cercle plein. Il s'agit de la valeur par défaut.
 - **circle** : le marqueur est un cercle creux.
 - **square** : le marqueur est un carré plein.

• Exemple :

Les noms des balises de bases à déclarer dans un document HTML sont:

- HTML
- HEAD
- BODY

Les noms des balises de bases à déclarer dans un document HTML sont:

```
<ul type="square">  
  <li>HTML</li>  
  <li>HEAD</li>  
  <li>BODY</li>  
</ul>
```

HTML 5 : « Présentation du langage HTML et notions de base »

✓ Les images :

▪ Balise : liste non ordonnée (ou numérotée)

- Permet d'afficher une image sur le navigateur.
- C'est une balise orpheline (donc pas de).

• Attribut « type » :

- **src**: la source de l'image. La valeur associée correspond au chemin vers elle (ou son emplacement). Le chemin peut être absolu ou relatif.

Le chemin absolu représente l'URL de l'image (par exemple: https://upload.wikimedia.org/wikipedia/commons/thumb/2/2f/Google_2015_logo.svg/2560px-Google_2015_logo.svg.png).

Le chemin relatif désigne l'emplacement de l'image à partir de l'emplacement actuel (par exemple: images/logo.png).

- **border**: accepte une valeur numérique (1,2...). Cette valeur désigne l'épaisseur de la bordure qui encadre l'image. L'unité utilisée pour la bordure est le pixel (px) mais elle est implicite, seule la valeur numérique est déclarée. Par défaut la bordure est de couleur noire.

HTML 5 : « Présentation du langage HTML et notions de base »

✓ Les images :

▪ Balise : liste non ordonnée (ou numérotée)

- Permet d'afficher une image sur le navigateur.
- C'est une balise orpheline (donc pas de).

• Attribut « type » :

- **width et height**: signifient, respectivement, la largeur et la hauteur de l'image. L'unité peut être le pixel ou le pourcentage (qu'il faut expliciter à l'aide du symbole universel %). Si par exemple on donne à la largeur la valeur 100%, cela veut dire que l'image occupe toute la largeur du conteneur qui accueille la balise .

Si width ou height ne sont pas spécifiée alors l'image sera affichée en taille réelle.

Si seule la valeur d'une dimension (width ou height) est déclarée, l'autre dimension sera automatiquement recalculée par le navigateur de telle sorte à conserver les proportions originales de l'image.

- **alt**: représente le texte alternatif à l'image.
- **title**: message affiché dans une infobulle si on survole l'image avec la souris.

HTML 5 : « Présentation du langage HTML et notions de base »

✓ Les images :

- Balise : liste non ordonnée (ou numérotée)
 - Permet d'afficher une image sur le navigateur.
 - C'est une balise orpheline (donc pas de).

```

```



HTML 5 : « Présentation du langage HTML et notions de base »

✓ Les liens hypertextes :

▪ Balise <a> :

- Un objet HTML (texte, image ou autre) sur lequel on peut cliquer pour aller vers une autre page (ou un autre emplacement dans la même page).
- Permet de relier les pages les unes aux autres pour constituer ainsi le Web.

• Attribut « href » :

- Désigne le chemin vers la page de destination après avoir cliqué sur le lien. Le chemin peut être absolu ou relatif.
- Si la valeur de l'attribut href vaut « <mailto:adresse@mail> », en cliquant sur le lien, le logiciel de messagerie par défaut installé chez le poste client s'ouvrira en l'invitant à écrire un mail à "adresse@mail".

• Attribut « target » :

- Désigne où la page de destination sera ouverte. Il peut contenir plusieurs valeurs mais une seule est fréquemment utilisée il s'agit de _blank. La valeur _blank signifie que le lien sera ouvert dans un nouvel onglet.
- Si l'attribut target n'est pas spécifié, alors la page de destination sera ouverte dans la même fenêtre.

HTML 5 : « Présentation du langage HTML et notions de base »

✓ Les liens hypertextes :

▪ Balise <a> :

- Un objet HTML (texte, image ou autre) sur lequel on peut cliquer pour aller vers une autre page (ou un autre emplacement dans la même page).
- Permet de relier les pages les unes aux autres pour constituer ainsi le Web.
- **Attribut « title » :** Décrit un message qui sera affiché dans une infobulle si on survole le lien avec la souris.

```
<a href="http://www.google.com" title="Aller vers google.com" target="_blank">
  Lien vers le site google.com
</a>
```

[Lien vers le site google.com](http://www.google.com)

HTML 5 : « Présentation du langage HTML et notions de base »

✓ Les liens hypertextes :

▪ Balise <a> : Lien internes

- Déplace vers un autre emplacement dans la même page déjà ouverte.
- Appelé « ancre ».
- **Solution 1 :**

```
<a name="toto"></a>
```

```
<a href="#toto">Aller vers toto</a>
```

Solution 2 :

```
<h2 id="section_further_down">Section plus  
bas</h2>
```

```
<p><a href="#section_further_down"> Passez à  
la rubrique ci-dessous </a></p>
```

HTML 5 : « Présentation du langage HTML et notions de base »

✓ Les tableaux :

- Grille qui contient des données structurées en ligne et en colonne.
- Conteneurs qui renferment d'autres objet HTML afin de structurer sa page Web sous forme de cellules et faciliter ainsi la mise en page.
- Pour créer un tableau nous avons besoin, au moins, de trois balises de base: <table>, <tr> et <td>.

- **Balise <table> :**

- Conteneur principal. Permet de déclarer un tableau.
- **Attribut « border » :** définit l'épaisseur de la bordure du tableau (et de ses cellules). Elle est exprimée en pixels (px) mais la valeur peut être déclarée sans l'unité.

Si le tableau est destiné à devenir conteneur d'autres objet HTML, l'attribut border aura donc la valeur 0.

- **Attribut « width » :** définit la largeur du tableau. Elle est exprimée en pixel ou en pourcentage. Si on entend les pixels alors peut mettre la valeur sans l'unité. Si c'est du pourcentage alors on peut mettre la valeur suivie du symbole %.

HTML 5 : « Présentation du langage HTML et notions de base »

✓ Les tableaux :

- Grille qui contient des données structurées en ligne et en colonne.
- Conteneurs qui renferment d'autres objet HTML afin de structurer sa page Web sous forme de cellules et faciliter ainsi la mise en page.
- Pour créer un tableau nous avons besoin, au moins, de trois balises de base: <table>, <tr> et <td>.
- **Balise <table>** :
 - Conteneur principal. Permet de déclarer un tableau.
 - **Attribut « height »** : définit la hauteur du tableau en pixel. Mais il vaut mieux ne pas déclarer cet attribut et laisser le tableau s'étirer en fonction de la taille de son contenu.
 - **Attribut « bgcolor »** : désigne la couleur de l'arrière-plan du tableau (en nom de la couleur ou en code de couleur).
 - **Attribut « cellspacing »** : signifie l'espacement entre les différentes cellules du tableau. Sa valeur est exprimée en pixel (implicite).
 - **Attribut « cellpadding »** : définit la marge interne des cellules. Sa valeur est exprimée en pixel (implicite).

HTML 5 : « Présentation du langage HTML et notions de base »

✓ Les tableaux :

- Grille qui contient des données structurées en ligne et en colonne.
- Conteneurs qui renferment d'autres objet HTML afin de structurer sa page Web sous forme de cellules et faciliter ainsi la mise en page.
- Pour créer un tableau nous avons besoin, au moins, de trois balises de base: <table>, <tr> et <td>.
- **Balise <tr>** :
 - Permet d'ajouter une ligne à un tableau.
 - La balise <tr> accepte des attributs comme height ou bgcolor, mais il est recommandé de ne pas utiliser d'attribut dessus.
- **Balise <td>** :
 - Représente une colonne. C'est elle en fait le vrai conteneur.
 - C'est la balise <td> qui doit avoir des attributs.

HTML 5 : « Présentation du langage HTML et notions de base »

✓ Les tableaux :

- Grille qui contient des données structurées en ligne et en colonne.
- Conteneurs qui renferment d'autres objet HTML afin de structurer sa page Web sous forme de cellules et faciliter ainsi la mise en page.
- Pour créer un tableau nous avons besoin, au moins, de trois balises de base: <table>, <tr> et <td>.
- Balise <td> :
 - Représente une colonne. C'est elle en fait le vrai conteneur.
 - C'est la balise <td> qui doit avoir des attributs.
 - Attribut « width » : définit la largeur de la colonne. Elle est exprimée en pixel ou en pourcentage.
 - Attribut « height » : définit la hauteur de la colonne en pixel.
 - Attribut « bgcolor » : désigne la couleur de l'arrière-plan de la colonne (en nom couleur ou en code de couleur).

HTML 5 : « Présentation du langage HTML et notions de base »

✓ Les tableaux :

- Grille qui contient des données structurées en ligne et en colonne.
- Conteneurs qui renferment d'autres objet HTML afin de structurer sa page Web sous forme de cellules et faciliter ainsi la mise en page.
- Pour créer un tableau nous avons besoin, au moins, de trois balises de base: <table>, <tr> et <td>.
- **Balise <td> :**
 - Représente une colonne. C'est elle en fait le vrai conteneur.
 - C'est la balise <td> qui doit avoir des attributs.
 - **Attribut « align » :** permet de spécifier l'alignement des objets contenus dans la colonne. Elle peut avoir une des valeurs: left (valeur par défaut), right, center ou justify.
 - **Attribut « valign » :** défini l'alignement vertical des objets contenus dans la colonne. Elle peut avoir une des valeurs: middle (valeur par défaut), top ou bottom.

HTML 5 : « Présentation du langage HTML et notions de base »

✓ Les tableaux :

```
<table border="1" width="100%" cellpadding="6">
  <tr>
    <td width="50%" bgcolor="#000000"><font color="#FFFFFF">Type de matériel</font>
  </td>
    <td width="50%" bgcolor="#000000"><font color="#FFFFFF">Quantité</font></td>
  </tr>
  <tr>
    <td width="50%">PC de bureau</td>
    <td width="50%">26</td>
  </tr>
  <tr>
    <td width="50%">PC portables</td>
    <td width="50%">32</td>
  </tr>
  <tr>
    <td width="50%">Imprimantes</td>
    <td width="50%">12</td>
  </tr>
</table>
```

Type de matériel	Quantité
PC de bureau	26
PC portables	32
Imprimantes	12

HTML 5 : « Présentation du langage HTML et notions de base »

✓ Les tableaux :

- **Balise <caption>** : déclarée directement après la balise <table>. Elle contient le titre du tableau.
- **Attribut « align »** : peut prendre une des valeurs suivantes : **top** (valeur par défaut) permet d'avoir le titre en dessus du tableau et **bottom** permet d'avoir le titre en dessous du tableau.
- **Balise <thead>** : permet de déclarer les cellules d'entête. Elle est déclarée dans la balise <table> et peut contenir des lignes (balise <th>) qui contiennent des colonnes (balise <td>). Les cellules d'entête sont incluses en haut.
- **Balise <th>** : désigne donc une légende pour les colonnes d'entête.
- **Balise <tbody>** : inclut les cellules du corps du tableau. <tbody> est déclarée dans la balise <table> est peut contenir les balises <tr> puis <td>.
- **Balise <tfoot>** : pied du tableau. <tfoot> est déclarée dans la balise <table> après la balise <thead> est peut contenir les balises <tr> puis <td>. Les cellules de la balise <tfoot> sont automatiquement intégrées à la fin du tableau.

HTML 5 : « Présentation du langage HTML et notions de base »

```
<table border="1" width="100%" cellspacing="0" cellpadding="6">
  <caption>Liste du matériel informatique disponible</caption>
  <thead>
    <tr>
      <th width="50%">Type de matériel</th>
      <th width="50%">Quantité</th>
    </tr>
  </thead>
  <tfoot>
    <tr>
      <th width="50%">Total</th>
      <th width="50%">70</th>
    </tr>
  </tfoot>
  <tbody>
    <tr>
      <td width="50%">PC de bureau</td>
      <td width="50%">26</td>
    </tr>
    <tr>
      <td width="50%">PC portables</td>
      <td width="50%">32</td>
    </tr>
    <tr>
      <td width="50%">Imprimantes</td>
      <td width="50%">12</td>
    </tr>
  </tbody>
</table>
```

Liste du matériel informatique disponible

Type de matériel	Quantité
PC de bureau	26
PC portables	32
Imprimantes	12
Total	70

HTML 5 : « Présentation du langage HTML et notions de base »

✓ Les tableaux :

- On peut fusionner les cellules horizontalement ou verticalement en utilisant des attributs sur la balise `<td>`.
- **Attribut « colspan » :** inclus dans la balise de colonne `<td>`. Il indique à celle-ci combien de colonnes il faut fusionner pour en avoir qu'une seule à la fin.
- **Attribut « rowspan » :** fusionne les cellules à la verticale.

Tableau sans fusion

Cellule 1	Cellule 2
Cellule 3	Cellule 4

Tableau avec fusion via rowspan

```
<table>
  <tr>
    <td rowspan="2">Cellule 1</td>
    <td>Cellule 2</td>
  </tr>
  <tr>
    <td>Cellule 4</td>
  </tr>
</table>
```

Cellule 1	Cellule 2
	Cellule 4

Tableau avec fusion via colspan

```
<table>
  <tr>
    <td colspan="2">Cellule 1</td>
  </tr>
  <tr>
    <td>Cellule 3</td>
    <td>Cellule 4</td>
  </tr>
</table>
```

Cellule 1	
Cellule 3	Cellule 4

HTML 5 : « Présentation du langage HTML et notions de base »

✓ Les formulaires :

- Permet de rendre la page Web plus interactive en la rendant capable de dialoguer avec l'internaute à travers des champs de saisie et boutons.
- Un formulaire permet à l'internaute de saisir du texte ou de valider un choix ou encore de sélectionner une entrée.
- Ces informations sont ensuite (dans la majorité des cas) envoyées vers l'application serveur pour les traiter. C'est très utile surtout quand il s'agit d'une page Web dynamique.
- **Balise <form>** : délimite les champs du formulaire (zones de texte, boutons...).
- **Attribut « name »** : permet d'identifier le formulaire par un nom. Comme toutes les nominations informatiques, ce nom ne doit contenir que des caractères conventionnels comme les lettres et les chiffres). Il est possible de déclarer plusieurs formulaires dans la même page à condition que chaque nom de formulaire soit unique.
- **Attribut « method »** : permet de spécifier la méthode à utiliser pour envoyer le formulaire au serveur. En effet, l'objectif d'un formulaire c'est de transférer les données saisies par le client à l'application serveur (ce qu'on appelle soumettre le formulaire). En HTML, il existe deux méthodes pour envoyer un formulaire: GET et POST.

HTML 5 : « Présentation du langage HTML et notions de base »

✓ Les formulaires :

- Permet de rendre la page Web plus interactive en la rendant capable de dialoguer avec l'internaute à travers des champs de saisie et boutons.
- Un formulaire permet à l'internaute de saisir du texte ou de valider un choix ou encore de sélectionner une entrée.
- Ces informations sont ensuite (dans la majorité des cas) envoyées vers l'application serveur pour les traiter. C'est très utile surtout quand il s'agit d'une page Web dynamique.
- **Balise <form>** : délimite les champs du formulaire (zones de texte, boutons...).
- **Attribut « method »** :
 - La méthode GET (method="get") : est la valeur par défaut. Cette méthode permet d'envoyer les données du formulaire à travers l'URL en utilisant les symboles ? pour préciser la suite de données et & pour séparer les données entre elles.
 - La méthode POST (method="post") : permet d'envoyer les données du formulaire vers l'application serveur à travers une entête, permet aussi de poster un volume de données plus important que celui transmis par GET.

HTML 5 : « Présentation du langage HTML et notions de base »

✓ Les formulaires :

- Permet de rendre la page Web plus interactive en la rendant capable de dialoguer avec l'internaute à travers des champs de saisie et boutons.
- Un formulaire permet à l'internaute de saisir du texte ou de valider un choix ou encore de sélectionner une entrée.
- Ces informations sont ensuite (dans la majorité des cas) envoyées vers l'application serveur pour les traiter. C'est très utile surtout quand il s'agit d'une page Web dynamique.
- **Balise <form>** : délimite les champs du formulaire (zones de texte, boutons...).
- **Attribut « action »** : désigne la page qui se chargera du traitement des données, envoyées du formulaire après sa soumission, vers l'application serveur pour le traitement. Si l'attribut action est vide ou n'est pas mentionné, alors c'est la page courante (celle qui contient le formulaire) qui sera la page action.

HTML 5 : « Présentation du langage HTML et notions de base »

✓ Les formulaires :

- **Balise <input>** : zone de texte qui permet d'écrire un texte sur une seule ligne. Il s'agit d'une balise orpheline (pas de </input>).
- **Attribut « type »** : permet de définir le type de champ à intégrer.
 - text : Permet d'avoir une zone de texte normale (adapté pour les noms, emails, login...)
 - password : Permet d'avoir une zone de mot de passe où les caractères sont masqués.
 - radio : un bouton radio. Il s'agit d'un petit cercle qu'on peut cocher et qui permet de faire un choix unique. Le fait de cocher un autre bouton radio décoche le premier.
 - checkbox : Permet de créer une case à cocher. C'est un petit carré qu'on peut cocher ou décocher. Il est destiné à faire des choix multiples.
 - file : Permet de créer le champ de chargement de fichier (comme une pièce jointe d'un email).
 - hidden : Pour créer un champ caché. C'est comme une zone de texte mais qui n'est pas visible sur le navigateur. Il sert à stocker provisoirement des données jusqu'à la phase de traitement.
 - button : Permet de créer un bouton simple. A la base, ce bouton ne fait aucune opération. Il faut donc le programmer (généralement avec Javascript).
 - reset : Permet de créer un bouton d'annulation. Si le formulaire contient des champs qu'on a déjà saisi, ce bouton permet de tout initialiser.
 - submit : C'est le bouton d'envoi. C'est lui qui permet de poster le formulaire vers la page définie dans l'attribut action de la balise <form>.

HTML 5 : « Présentation du langage HTML et notions de base »

✓ Les formulaires :

- **Balise <input>** : zone de texte qui permet d'écrire un texte sur une seule ligne. Il s'agit d'une balise orpheline (pas de </input>).
- **Attribut « name »** : permet de donner un nom au champ de formulaire. Le nom sert d'identifiant du champ et doit être unique.
- **Attribut « value »** : L'attribut value sert à définir la valeur par défaut d'un champ.
S'il s'agit d'une zone de texte, le fait de déclarer l'attribut value force le navigateur à initialiser ce champ avec le texte faisant office de valeur de l'attribut.
Si le champ est un bouton, alors le texte de l'attribut value constitue l'étiquette (ou label). C'est le texte visible sur le bouton.
Pour les boutons radio ou les cases à cocher, le texte de l'attribut value n'est pas visible sur le navigateur mais il est utile (si on fait appel à des programmes comme Javascript ou PHP).
- **Attribut « size »** : contient un entier comme valeur. Il définit la largeur du champ en caractères. Par défaut, une zone de texte mesure 20 caractères (cela veut dire qu'on peut voir jusqu'à 20 caractères à la fois même si le texte est plus grand).

HTML 5 : « Présentation du langage HTML et notions de base »

✓ Les formulaires :

- **Balise <input>** : zone de texte qui permet d'écrire un texte sur une seule ligne. Il s'agit d'une balise orpheline (pas de </input>).
- **Attribut « tabindex »** : permet de définir l'ordre de tabulation. En pratique, si on veut saisir un grand formulaire, on se déplace d'un champ à un autre en appuyant sur la touche "tabulation" du clavier. Par défaut le curseur se déplace dans l'ordre où sont déclarés les champs. On peut rompre cet ordre en définissant les valeurs de tabindex. La valeur est un entier qui commence de 1. Le curseur se déplacera dans l'ordre croissant de tabindex.
- **Attribut « checked »** : un attribut booléen. Explicitement il est déclaré comme ceci: checked="true", mais il suffit de déclarer checked tout seul pour que le navigateur sache qu'il est activé. Cet attribut s'applique uniquement sur les boutons radio (type="radio") et les cases à cocher (type="checkbox"). S'il est déclaré alors le bouton est automatiquement coché au chargement de la page.

HTML 5 : « Présentation du langage HTML et notions de base »

✓ Les formulaires :

- **Balise <select>** : permet de déclarer une liste de sélection (ou liste déroulante).

Dispose des attributs **name** et **tabindex**.

Pour remplir cette liste avec les valeurs (appelées options) on doit déclarer une autre balise à l'intérieur, c'est la balise **<option>**.

La balise **<option>** dispose d'un attribut booléen du nom de **selected**. Si cet attribut est déclaré, c'est l'option qui le contient qui sera automatiquement sélectionnée au chargement de la page.

```
<form name="inscription" method="post" action="">
  Civilité<br>
  <input type="radio" name="civilite"> Mlle<br>
  <input type="radio" name="civilite"> Mme<br>
  <input type="radio" name="civilite"> M.<br>
  Nom et prénom<br>
  <input type="text" name="nom" value=""><br>
  Email<br>
  <input type="text" name="email" value=""><br>
  Login<br>
  <input type="text" name="login" value=""><br>
  Mot de passe<br>
  <input type="password" name="pass" value=""><br>
  Vous être<br>
  <select name="niv">
    <option>Etudiant</option>
    <option>Fonctionnaire</option>
    <option>Employé au secteur privé</option>
  </select><br>
  <input type="button" name="envoyer" value="S'inscrire">
</form>
```

Civilité
☐ Mlle
☐ Mme
☐ M.

Nom et prénom

Email

Login

Mot de passe

Vous être

HTML 5 : « Présentation du langage HTML et notions de base »

✓ Les formulaires :

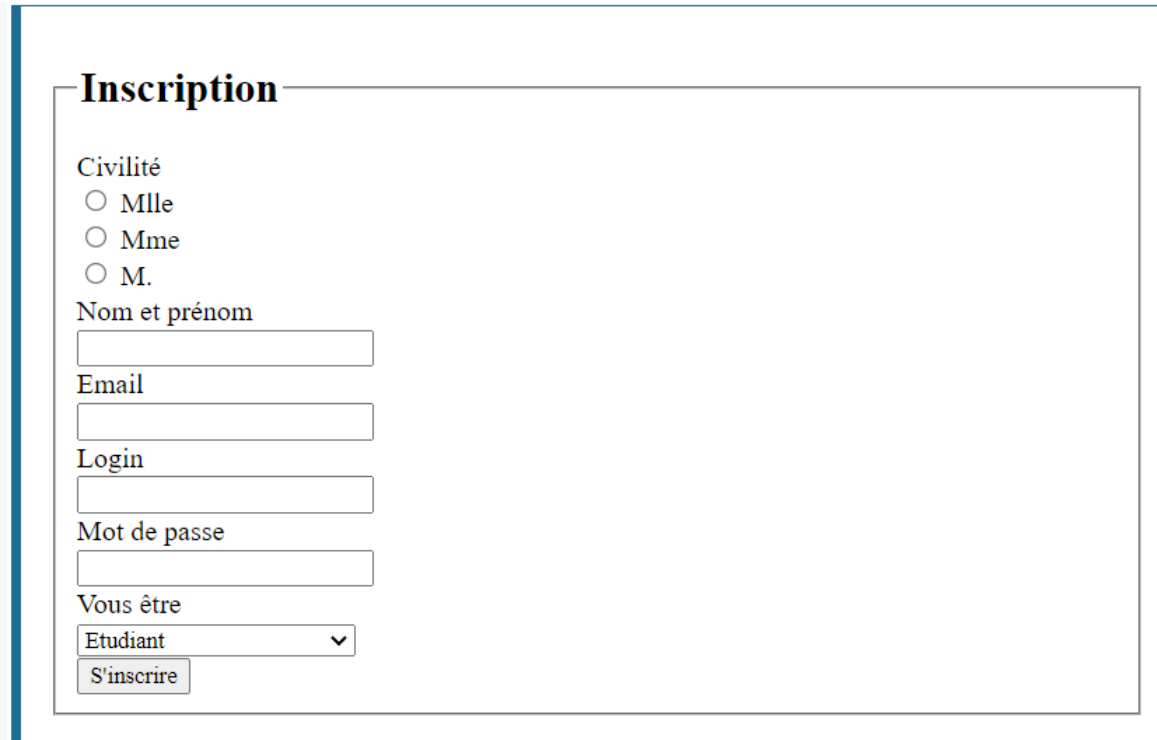
- **Balise <textarea>** : permet de définir un espace de texte. Un espace de texte est une grande zone de texte qui permet d'écrire des paragraphes entiers.
- **Attribut « cols »** : définit la largeur en caractères de l'espace de texte. Elle désigne combien de caractère une ligne peut-elle contenir. Le fait de dépasser la largeur définie entraîne un retour automatique à la ligne.
- **Attribut « rows »** : définit la hauteur de l'espace de texte. La hauteur désigne combien de ligne on peut voir à la fois. Si le texte dépasse le nombre de lignes défini, alors une barre de défilement qui permet d'atteindre le reste des lignes apparaît.

HTML 5 : « Présentation du langage HTML et notions de base »

✓ Les formulaires :

- **Balise <fieldset>** : permet d'appliquer un cadre au formulaire avec une légende en guise de titre avec la balise <legend>.

```
<form name="inscription" method="post" action="">
  <fieldset>
    <legend><h2>Inscription</h2></legend>
    Civilité<br>
    <input type="radio" name="civilite"> Mlle<br>
    <input type="radio" name="civilite"> Mme<br>
    <input type="radio" name="civilite"> M.<br>
    Nom et prénom<br>
    <input type="text" name="nom" value=""><br>
    Email<br>
    <input type="text" name="email" value=""><br>
    Login<br>
    <input type="text" name="login" value=""><br>
    Mot de passe<br>
    <input type="password" name="pass" value=""><br>
    Vous être<br>
    <select name="niv">
      <option>Etudiant</option>
      <option>Fonctionnaire</option>
      <option>Employé au secteur privé</option>
    </select><br>
    <input type="button" name="envoyer" value="S'inscrire">
  </fieldset>
</form>
```



Inscription

Civilité

☐ Mlle

☐ Mme

☐ M.

Nom et prénom

Email

Login

Mot de passe

Vous être

HTML 5 : « Balises spéciales: marquee, iframe et object »

- **Balise <marquee>** : permet de faire défiler un objet HTML (texte, image...).
- **Attribut « behavior »** : agit sur le style d'animation souhaité. Il peut avoir trois valeurs différentes :
 - scroll: C'est le comportement par défaut. Il permet de faire défiler l'objet d'un côté à un autre. Quand l'objet disparaît, il réapparaît à l'endroit initial et refait le cycle.
 - slide: L'objet défile d'un côté vers l'autre, quand il atteint la position finale il s'arrête et l'animation est interrompue.
 - alternate: L'objet défile en va-et-vient entre la position 1 et la position 2.

```
<marquee behavior="alternate">  
    Texte défilant en va-et-vient  
</marquee>
```

HTML 5 : « Balises spéciales: marquee, iframe et object »

- **Balise <marquee>** : permet de faire défiler un objet HTML (texte, image...).
- **Attribut « behavior »** : agit sur le style d'animation souhaité. Il peut avoir trois valeurs différentes :
 - scroll: C'est le comportement par défaut. Il permet de faire défiler l'objet d'un côté à un autre. Quand l'objet disparaît, il réapparaît à l'endroit initial et refait le cycle.
 - slide: L'objet défile d'un côté vers l'autre, quand il atteint la position finale il s'arrête et l'animation est interrompue.
 - alternate: L'objet défile en va-et-vient entre la position 1 et la position 2.
- **Les attributs width et height** : désignent respectivement la largeur et la hauteur de la zone de défilement. Ils sont exprimés en pixel (px) qui est implicite (la valeur est déclarée sans l'unité).
- **L'attribut bgcolor** : spécifie la couleur d'arrière-plan de la zone de défilement. Elle est exprimée en nom anglais de la couleur ou code de celle-ci.
- **L'attribut direction** : spécifie la direction du défilement. Elle peut avoir 4 valeurs:
 - left: Signifie vers la gauche. L'objet défile de droite à gauche.
 - right: L'objet défile de gauche à droite.
 - up: L'objet défile du bas vers le haut
 - down: L'objet défile du haut vers le bas

```
<marquee behavior="scroll" direction="left" width="100%"  
  bgcolor="#EEEEEE">  
  <h2>Texte défilant horizontalement</h2>  
</marquee><br>  
<marquee behavior="scroll" direction="down" width="100%"  
  height="200" bgcolor="#EEEEEE">  
  <h2>Texte défilant verticalement</h2>  
</marquee>
```

HTML 5 : « Balises spéciales: marquee, iframe et object »

- **Balise <marquee>** : permet de faire défiler un objet HTML (texte, image...).
- **Attribut scrollamount** : signifie le pas de défilement. Entre l'instant t et t+1 l'objet parcourt une distance appelée pas de défilement. Plus le pas est grand plus le défilement aura l'air rapide et inversement. Il accepte une valeur numérique qui désigne le nombre de pixels à parcourir.
- **Attribut scrolldelay** : signifie le délai de défilement. Il s'agit de la durée pendant laquelle l'objet reste dans la même position. Plus il est grand plus le défilement est lent. Il a comme valeur un nombre entier qui désigne le nombre de millisecondes (l'unité ms est implicite).
- **Attribut loop** : indique combien de fois le défilement sera répété. Par défaut il vaut la valeur -1 qui signifie à l'infini.

HTML 5 : « Balises spéciales: marquee, iframe et object »

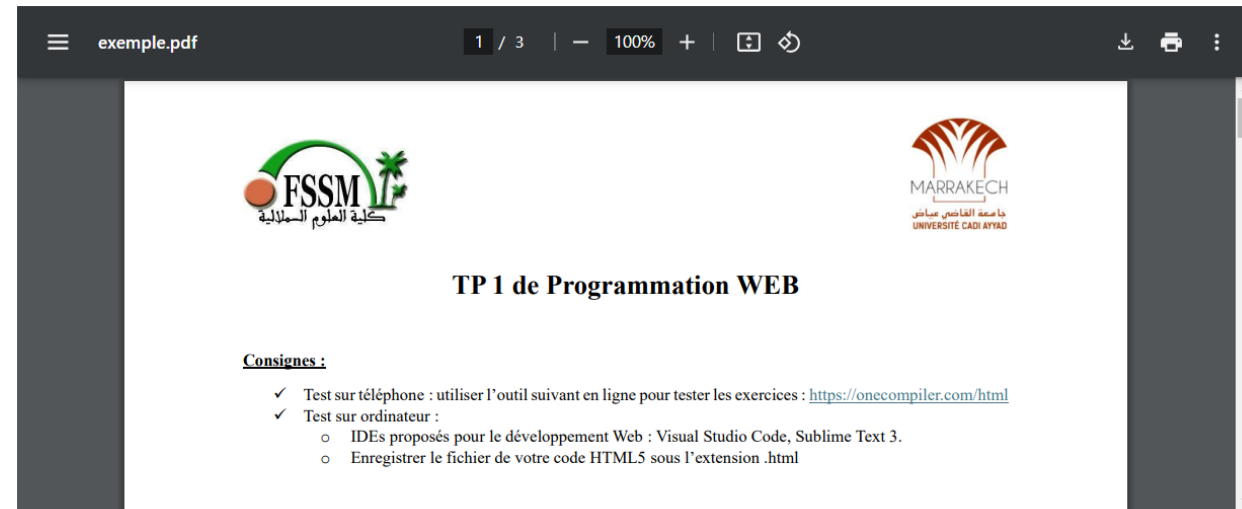
- **Balise <iframe>** : permet d'insérer une page web au sein d'une autre. Elle peut être insérée n'importe où entre les balises <body> et </body>.
- **Attribut src** : permet d'indiquer le chemin (ou l'URL) de la page web à insérer dans la balise <iframe>. Il n'est pas nécessaire que la page à insérer appartienne au même site Web. On peut toujours appeler une page appartenant à autre site.
- **Attributs width et height** : permettent d'indiquer, respectivement, la largeur et la hauteur de l'iframe.
- **Attribut frameborder** : permet de spécifier l'épaisseur de la bordure de l'iframe. Elle est utilisée exactement comme l'attribut border sur les images ou les tableaux.
- **Attribut scrolling** : permet d'afficher une barre de défilement sur l'iframe si la page appelée est plus grande que les dimensions de celle-ci.
- **Attribut name** : L'attribut name permet d'identifier une iframe par un nom unique.



HTML 5 : « Balises spéciales: marquee, iframe et object »

- **Balise <object>** : permet au navigateur d'afficher tout type de documents comme du contenu multimédia (audio ou vidéo) ou encore des documents PDF...
- **Attributs width et height** : spécifient la largeur et la hauteur de la zone d'affichage de la balise <object>.
- **Attribut type** : indique le type de données à insérer dans la balise <object> comme par exemple text/html qui signifie document texte écrit en HTML ou encore application/pdf qui désigne un document PDF.
- **Attribut data** : spécifie le chemin (ou URL) du contenu à insérer dans la balise <object>.

```
<object data="docs/ExemplePDF.pdf" type="application/pdf"
width="100%" height="400">
  <a href="docs/ExemplePDF.pdf">Le PDF</a>
</object>
```



HTML 5 : « Balises META »

✓ Balise <meta> :

- Contiennent les métadonnées. Il s'agit de données supplémentaires qui aident à l'exécution correcte de la page ou fournissent des informations additionnelles lors de l'exécution.
- Les balises <meta> sont déclarées dans l'entête du document HTML (entre les balises <head> et </head>). Son contenu n'est donc pas affiché sur la page.
- C'est une balise orpheline (pas de </meta>).
- La balise META contient en général deux attributs. Le deuxième est toujours l'attribut content. Quant au premier il peut être soit http-equiv ou name :
 - **Balise META avec l'attribut http-equiv** : fournit des informations supplémentaires au navigateur exécutant la page, comme l'encodage ou les redirections...
 - **Balise META avec l'attribut name** : fournit des informations complémentaires aux outils de recherche et d'indexation (comme les moteurs de recherche). Elles sont largement utilisées pour le référencement des sites Web.

HTML 5 : « Balises META »

✓ **Balise <meta> :**

- Les balises META les plus utilisées :
- **META Content-Type** : permet de spécifier le type du document qui la contient et son encodage. Un document HTML est un document texte. L'encodage universel de caractères qui couvre tous les systèmes d'écriture est UTF-8.

```
<meta http-equiv="Content-Type" content="text/html; charset=UTF-8">
```

- **META refresh** : permet de rafraîchir la page ou rediriger le navigateur vers une autre page.

```
<meta http-equiv="refresh" content="3;url=http://www.w3.org">
```

Si la page qui contient ce code est chargée sur le navigateur, celui-ci sera redirigé vers le site de W3C après une durée d'attente de 3 secondes.

Si la durée n'est pas spécifiée, alors elle vaut 0 seconde et la redirection est instantanée.

Si la partie contenant le message url=http://www.w3.org n'est pas déclarée, alors c'est la page courante qui sera rafraîchie.

HTML 5 : « Balises META »

✓ **Balise <meta> :**

- Les balises META les plus utilisées :
- **META Content-Language** : permet de préciser la langue principale de la page Web.

```
<meta http-equiv="Content-Language" content="fr-FR">
```

- **META description** : intéresse principalement les outils de recherche (comme les moteurs de recherche). Elle contient un résumé de la page courante. Utile pour le référencement, c'est ce qui explique le soin pris par les développeurs d'y déclarer les mots clés (pertinents) susceptibles d'attirer les internautes vers leurs sites. Le résumé doit être concis (deux ou trois phrases pertinentes suffisent).

```
<meta name="description" content="résumé de la page Web">
```

HTML 5 : « Balises META »

✓ **Balise <meta> :**

- Les balises META les plus utilisées :
- **META Keywords** : renferme les mots-clés qui ont un rapport avec le contenu de la page. Elle est de moins en moins prise en considération par les outils de recherche.

```
<meta name="keywords" content="mot clé 1, mot clé 2, mot clé 3, ...">
```

- **META generator** : permet d'indiquer l'outil utilisé pour créer le site Web. Généralement utilisé par les CMS (Content Management System). Sa déclaration permet aux auteurs de ces logiciels de faire des statistiques sur le taux d'utilisation de leurs outils par les créateurs des sites Web. Cette balise n'est pas utile pour le référencement.

```
<meta name="generator" content="nom et version du logiciel">
```

HTML 5 : « Validation et compatibilité »

- ✓ Une page Web ne s'affiche pas toujours de la même manière d'un navigateur à un autre.
- ✓ Solutions :
 - **Validation de la page** : via le validateur de W3C dont le lien est validator.w3.org afin de rendre le code conforme aux spécifications du Consortium.
 - **Faire des tests sur des moteurs de rendu différents** : Chaque navigateur embarque un moteur de rendu. Parmi ces derniers on trouve Gecko, Trident, Webkit, Blink, Presto et bien d'autres. Les intégrateurs choisissent alors un navigateur par moteur de rendu pour faire les tests afin de garantir un rendu uniforme de la page.
 - Webkit (Safari),
 - Blink (Chromium et dérivés : Google Chrome
 - Gecko (Mozilla Firefox)
 - ...

CSS : « Introduction aux feuilles de style »

✓ CSS : Cascading Style Sheets

- Il s'agit d'un langage de style avec une simple syntaxe et un rendement remarquable.
- Le CSS s'intéresse à la mise en forme du contenu intégré avec du HTML.
- Il y a plusieurs façons d'intégrer un style CSS :
- **Attribut de style local :**

```
<font style="color:red; font-family:verdana; font-size:12pt">  
  Bonjour à tous!  
</font>
```

Bonjour à tous!

- **Inconvénient :** il ne s'applique qu'à la balise sur laquelle il est déclaré

CSS : « Introduction aux feuilles de style »

✓ CSS : Cascading Style Sheets

- Il s'agit d'un langage de style avec une simple syntaxe et un rendement remarquable.
- Le CSS s'intéresse à la mise en forme du contenu intégré avec du HTML.
- Il y a plusieurs façons d'intégrer un style CSS :
- Style global à la page :

Bonjour à tous!

```
<!DOCTYPE html>
<html>
  <head>
    <meta charset="UTF-8" />
    <style>
      font{
        color:red;
        font-family:verdana;
        font-size:12pt;
      }
    </style>
  </head>
  <body>
    <font>
      Bonjour à tous!
    </font>
  </body>
</html>
```

CSS : « Introduction aux feuilles de style »

✓ CSS : Cascading Style Sheets

- Il s'agit d'un langage de style avec une simple syntaxe et un rendement remarquable.
- Le CSS s'intéresse à la mise en forme du contenu intégré avec du HTML.
- Il y a plusieurs façons d'intégrer un style CSS :
- La feuille de style en cascade :

```
<!DOCTYPE html>
<html>
  <head>
    <meta charset="UTF-8" />
    <link rel="stylesheet" type="text/css" href="style.css" />
  </head>
  <body>
    <h1>Exemple de CSS</h1>
    <font>
      Voici à quoi ressemble l'effet de la feuille de style
      sur ce document.
    </font>
  </body>
</html>
```

Fichier style.css

```
font{
  color:red;
  font-family:verdana;
  font-size:12pt;
}
h1{
  color:blue;
}
```

CSS : « Introduction aux feuilles de style »

✓ CSS : Cascading Style Sheets

- Il s'agit d'un langage de style avec une simple syntaxe et un rendement remarquable.
- Le CSS s'intéresse à la mise en forme du contenu intégré avec du HTML.
- Il y a plusieurs façons d'intégrer un style CSS :
- **La feuille de style en cascade :**
- **rel** : permet de définir le type de ressource à intégrer au document. La balise <link> peut aussi intégrer d'autres types de ressources, notamment les icones (connus sous le nom favicon).
- **type** : facultatif qui permet de spécifier le type et le MIME de la ressource à intégrer. Dans ce cas il s'agit de text/css qui désigne: document texte écrit en CSS.
- **href** : permet de spécifier le chemin (relatif ou absolu) de la feuille de style à appliquer.

```
<!DOCTYPE html>
<html>
  <head>
    <meta charset="UTF-8" />
    <link rel="stylesheet" type="text/css" href="style.css" />
  </head>
  <body>
    <h1>Exemple de CSS</h1>
    <font>
```

Voici à quoi ressemble l'effet de la feuille de style sur ce document.

```
</font>
```

PR. SOUKAINA MIAHED

Fichier style.css

```
font{
  color:red;
  font-family:verdana;
  font-size:12pt;
}
h1{
  color:blue;
}
```

CSS : « Introduction aux feuilles de style »

✓ CSS : Cascading Style Sheets

- Il s'agit d'un langage de style avec une simple syntaxe et un rendement remarquable.
- Le CSS s'intéresse à la mise en forme du contenu intégré avec du HTML.
- Il y a plusieurs façons d'intégrer un style CSS :
- La feuille de style en cascade :

```
<!DOCTYPE html>
<html>
  <head>
    <meta charset="UTF-8" />
    <link rel="stylesheet" type="text/css" href="style.css" />
  </head>
  <body>
    <h1>Exemple de CSS</h1>
    <font>
      Voici à quoi ressemble l'effet de la feuille de style
      sur ce document.
    </font>
  </body>
</html>
```

Fichier style.css

```
font{
  color:red;
  font-family:verdana;
  font-size:12pt;
}
h1{
  color:blue;
}
```

Exemple de CSS

Voici à quoi ressemble l'effet de la feuille de style sur ce document.

CSS : « Couleurs de textes et propriétés agissant sur le fond »

- ✓ **Commentaire** : `/* votre commentaire */`.
- ✓ **Couleur de texte** : permet de spécifier la couleur à appliquer à un texte. Le texte peut être placé dans n'importe quelle balise (`<div>`, `<span>`, `<header>`...).
- ✓ **Exemple** :
 - Nom de la couleur en anglais.
 - Code hexadécimal.
 - Code décimal (ou RGB).
 - Code RGBA : le quatrième chiffre indique le degré de transparence de la couleur appliquée. Il est compris entre 0 (pour totalement transparent) et 1 (pour totalement opaque)
 - Le mot clé `inherit` : permet à une balise d'hériter le style de la balise dans laquelle elle est incluse.

```
h1{
    color:orange;
}
div{
    color:#888888;
}
span{
    color:rgb(0,0,255);
}
footer{
    color:rgba(0,0,0,0.5);
}
```

CSS : « Couleurs de textes et propriétés agissant sur le fond »

- ✓ **Commentaire** : `/* votre commentaire */`.
- ✓ **Couleur de texte** : permet de spécifier la couleur à appliquer à un texte. Le texte peut être placé dans n'importe quelle balise (`<div>`, `<span>`, `<header>`...).
- ✓ **Exemple** :
 - Nom de la couleur en anglais.
 - Code hexadécimal.
 - Code décimal (ou RGB).
 - Code RGBA : le quatrième chiffre indique le degré de transparence de la couleur appliquée. Il est compris entre 0 (pour totalement transparent) et 1 (pour totalement opaque)
 - Le mot clé `inherit` : permet à une balise d'hériter le style de la balise dans laquelle elle est incluse.

```
h1{
    color:orange;
}
div{
    color:#888888;
}
span{
    color:rgb(0,0,255);
}
footer{
    color:rgba(0,0,0,0.5);
}
```

CSS : « Couleurs de textes et propriétés agissant sur le fond »

✓ Propriétés de l'arrière-plan :

- **background-color** : applique une couleur d'arrière-plan à l'objet HTML désigné.

▪ Exemple :

```
body{  
    background-color:#CCCCCC;  
}
```

- **background-image** : sert à insérer une image en arrière-plan de l'objet désigné.
- **Syntaxe** : background-image:url("chemin_de_l_image")
- Le chemin de l'image peut être absolu ou relatif à partir de l'emplacement où est déclaré le style.
- L'image est appliquée en arrière-plan avec sa taille réelle.
- Si elle est plus grande que l'objet qui l'accueille alors elle sera tronquée.
- Si elle est plus petite alors elle sera répétée jusqu'à ce que tout le fond de l'objet soit couvert.

```
body{  
    background-image:url("images/footerlogo.png");  
}
```



CSS : « Couleurs de textes et propriétés agissant sur le fond »

✓ Propriétés de l'arrière-plan :

- **background-repeat** : permet de contrôler la répétition de l'image au cas où celle-ci est plus petite que l'objet qui l'accueille.
- repeat: C'est la valeur par défaut. Elle permet de répéter l'image sur la largeur et la hauteur de l'objet.
- repeat-x: Sert à répéter l'image seulement sur la largeur de l'objet.
- repeat-y: Sert à répéter l'image seulement sur la hauteur de l'objet.
- no-repeat: Sans répétition. Par défaut elle sera placée sur le haut à gauche de l'objet.

```
body{  
    background-image:url("images/footerlogo.png");  
    background-repeat:repeat-x;  
}
```



CSS : « Couleurs de textes et propriétés agissant sur le fond »

✓ Propriétés de l'arrière-plan :

- **background-position** : permet de placer l'image de l'arrière-plan à l'endroit souhaité de l'objet.
- La valeur que l'on peut donner à cette propriété peut être exprimé en pourcentage (%) ou par les mots clé: top, center, bottom, left et right.
- **Exemple** : placer l'image d'arrière-plan au juste milieu de l'objet via **"background-position:center center"** ou **"background-position:50% 50%"**. La première valeur est la largeur et la deuxième pour la hauteur.

```
body{  
    background-image:url("images/footerlogo.png");  
    background-repeat:no-repeat;  
    background-position:50% 50%;  
}
```



CSS : « Couleurs de textes et propriétés agissant sur le fond »

✓ Propriétés de l'arrière-plan :

- **background-attachement** : permet de fixer l'arrière-plan de l'objet si celui-là contient une barre de défilement.
- Elle peut avoir deux valeurs:
- scroll qui est la valeur par défaut et qui laisse défiler l'arrière-plan avec l'avant plan si le client entraîne la barre de défilement qui apparaît sur l'objet
- fixed qui permet de fixer l'arrière-plan pendant que l'avant plan glisse en dessus.
- **Exemple** : placer l'image d'arrière-plan au juste milieu de l'objet via "background-position:center center" ou "background-position:50% 50%". La première valeur est la largeur et la deuxième pour la hauteur.

```
body{
    background-image:url("images/footerlogo.png");
    background-repeat:no-repeat;
    background-position:50% 50%;
    background-attachment:fixed;
}
```

CSS : « Couleurs de textes et propriétés agissant sur le fond »

✓ Propriétés de l'arrière-plan :

- **background-size** : permet de modifier la taille de l'image appliquée en arrière-plan d'un container (ou de la page entière). Par défaut, l'image est appliquée avec sa taille réelle.
- Peut accepter deux valeurs : la largeur et la hauteur. Ces valeurs peuvent être exprimées en pixel ou en pourcentage.
- La valeur "**cover**" permet d'adapter l'image au container de telle sorte à en exploiter l'espace maximal.
- **Exemple :**

```
body{  
    background-image:url("images/footerlogo.png");  
    background-size:cover;  
}
```

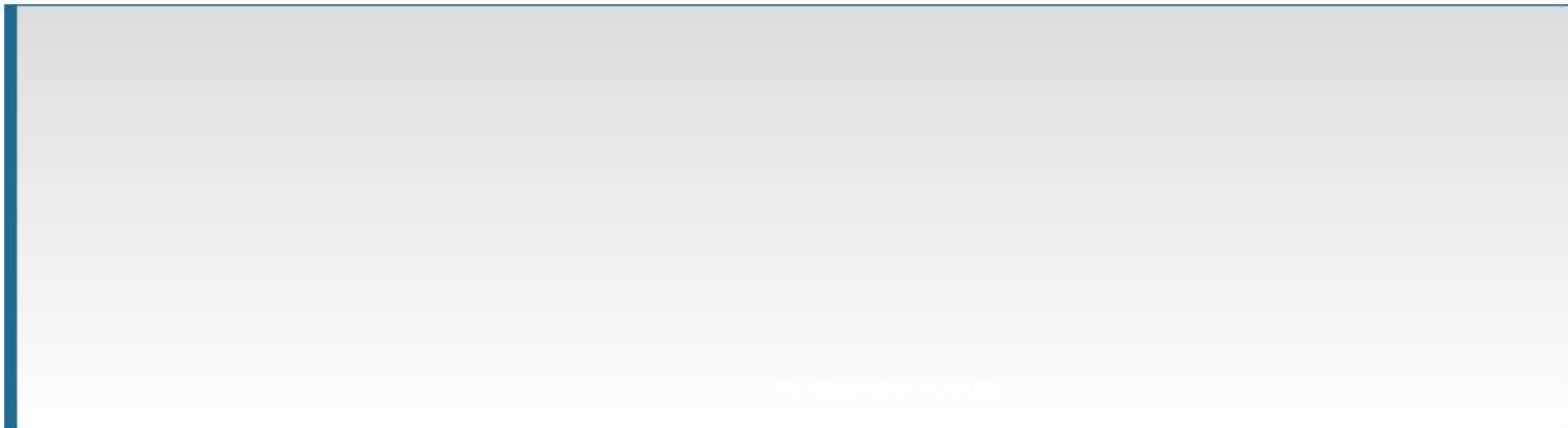


CSS : « Couleurs de textes et propriétés agissant sur le fond »

✓ Propriétés de l'arrière-plan :

- **linear-gradient** : s'applique à la propriété background pour générer un arrière-plan en dégradé de couleur :
- Direction du dégradé: qui peut avoir les valeurs comme to bottom (vers le bas), to left (vers la gauche), to left bottom (incliné vers la gauche et vers le bas) etc... Par défaut elle vaut la valeur to bottom.
- Couleur de début: Précise à partir de quelle couleur le dégradé sera engendré.
- Couleur de fin: Précise la deuxième couleur du dégradé.
- **Exemple :**

```
body{  
    background:linear-gradient(to bottom,#DDDDDD,#FFFFFF);  
}
```



CSS : « Couleurs de textes et propriétés agissant sur le fond »

✓ Propriétés de l'arrière-plan :

▪ linear-gradient :

- La syntaxe de la sous propriété linear-gradient est légèrement modifiée pour les anciennes versions de certains navigateurs afin de la rendre compatible. En effet, on la précède par un préfixe connu sous le nom de préfixe vendeur. Ce préfixe fait référence aux différents moteurs de rendu existants :
- -moz-: signifie Mozilla et fait référence à son moteur de rendu Gecko.
- -webkit-: signifie le moteur de rendu Webkit (utilisé par Google Chrome et Safari).
- -o-: signifie Opera et son moteur de rendu Presto.
- -ms-: signifie Microsoft et fait référence à son moteur de rendu Trident.
- **Exemple :**

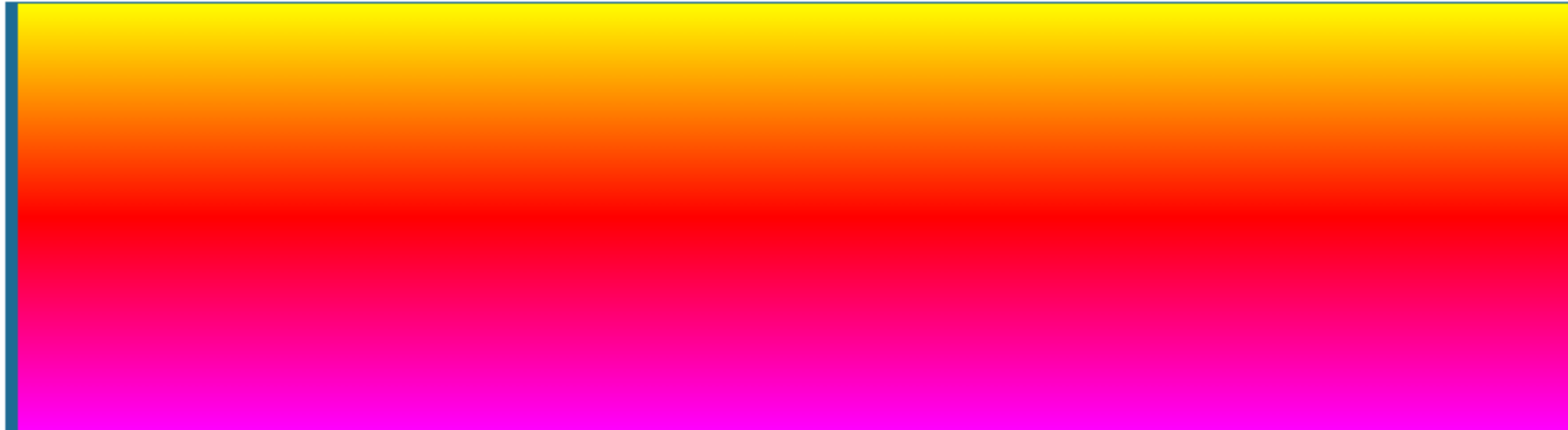
```
body{  
    -moz-background:linear-gradient(to bottom,#DDDDDD,#FFFFFF);  
    -webkit-background:linear-gradient(to bottom,#DDDDDD,#FFFFFF);  
    -o-background:linear-gradient(to bottom,#DDDDDD,#FFFFFF);  
    -ms-background:linear-gradient(to bottom,#DDDDDD,#FFFFFF);  
    background:linear-gradient(to bottom,#DDDDDD,#FFFFFF);  
}
```

CSS : « Couleurs de textes et propriétés agissant sur le fond »

✓ Propriétés de l'arrière-plan :

- **linear-gradient :**
- Il est possible de spécifier plusieurs couleurs pour le dégradé. Le navigateur fera en sorte de répartir les espaces d'une manière équitable.
- **Exemple :**

```
body{  
    background:linear-gradient(to bottom,#FF0,#F00,#F0F);  
}
```



CSS : « Couleurs de textes et propriétés agissant sur le fond »

✓ Propriétés de l'arrière-plan :

- **linear-gradient :**
- Il est possible de répartir les dégradés non équitablement en spécifiant la longueur d'étalement. Il est préférable de spécifier cette longueur en pourcentage.
- **Exemple :**

```
body{  
    background:linear-gradient(to bottom,#FF0 0%,#F00 30%,#F0F 100%);  
}
```



CSS : « Polices »

✓ Propriétés de font :

- **Font-size** : permet de spécifier la taille de la police. Si cette propriété n'est pas spécifiée alors la taille de la police vaut par défaut 12 points.
- L'unité px : Il s'agit d'une taille absolue. px désigne pixel. La taille de la police désigne la hauteur des caractères en pixel.
- L'unité pt : une taille absolue. pt désigne point. C'est une unité connue en typographie. 1 point mesure 1/72 pouces.
- Les unités conventionnelles : cm, mm, in (pour inch ou pouce)...
- L'unité em : em est une unité relative. 1em désigne la taille normale. Pour agrandir la taille alors il faut déclarer une valeur supérieure à 1 (par exemple 1.2em) et inversement.
- L'unité % : Le pourcentage est également une taille relative. 100% désigne la taille normale et 80% une taille plus petite.

CSS : « Polices »

✓ Propriétés de font :

- **Font-style** : permet d'écrire un texte en italique. Par défaut elle vaut la valeur normale qui n'a aucun effet sur le texte. La valeur italic incline le texte.

```
span{  
    font-style:italic;  
}
```

- **font-variant** : permet de convertir le texte en petites majuscules. Par défaut elle vaut la valeur normale. Si la valeur small-caps est déclarée alors la police sera transformé.

```
h1{  
    font-variant:small-caps;  
}
```

TITRE H1 CONVERTI EN PETITES MAJUSCULES.

CSS : « Polices »

✓ Propriétés de font :

- **Font-weight** : désigne le niveau du gras de la police.
- Les valeurs les plus utilisées sont : **bold** qui met le texte en gras (comme le fait la balise) et **normal** qui désactive le gras si le texte l'était déjà (cas des balises <h1>, <h2>... par exemple).

```
h1{  
  font-weight:normal;  
}
```

Titre h1 sans graisse.

- **font-family** : désigne la police d'écriture du texte. Elle accepte comme valeur le nom de la police.

```
body{  
  font-family:verdana;  
}
```

```
body{  
  font-family:"Century Gothic";  
}
```

CSS : « Polices »

✓ Propriétés de font :

- **font-family** : désigne la police d'écriture du texte. Elle accepte comme valeur le nom de la police.
- Si la police choisie n'est pas disponible, le navigateur chargera la police par défaut qui est "Times New Roman" qui n'est pas adaptée aux sites Web. La solution consiste donc à déclarer une liste de polices et laisser le navigateur la parcourir jusqu'à ce qu'il rencontre une police disponible.

```
body{  
    font-family:"Century Gothic", verdana, sans-serif;  
}
```

- Il existe plusieurs familles génériques de police, mais sur le Web nous n'avons recours qu'à trois d'entre elles:
- Famille de police avec empattement (Sérf) : comme "Times New Roman". Le nom CSS de cette famille est serif (sans accent).
- Famille de police sans empattement (Sans Sérf) : comme "verdana" ou "arial". Le nom CSS de cette famille est sans-serif.
- Famille de police à chasse fixe (Monospace) : comme "Courier" et "Courier New". Le nom CSS correspondant est monospace.

CSS : « Polices »

✓ Propriétés de font :

- **font-family** : désigne la police d'écriture du texte. Elle accepte comme valeur le nom de la police.
- Il existe plusieurs familles génériques de police, mais sur le Web nous n'avons recours qu'à trois d'entre elles:
- Famille de police avec empattement (Sérf) : comme "Times New Roman". Le nom CSS de cette famille est serif (sans accent).
- Famille de police sans empattement (Sans Sérf) : comme "verdana" ou "arial". Le nom CSS de cette famille est sans-serif.
- Famille de police à chasse fixe (Monospace) : comme "Courier" et "Courier New". Le nom CSS correspondant est monospace.

```
<h1 style="font-weight:normal; font-family:serif">MIME</h1>  
<h1 style="font-weight:normal; font-family:sans-serif">MIME</h1>  
<h1 style="font-weight:normal; font-family:monospace">MIME</h1>
```

Code source

Code source

Code source

CSS : « Polices »

✓ Propriétés de font :

- **Police embarquée** : ou polices en ligne. Il s'agit d'une police qui est envoyée au client par le serveur pour s'en servir durant l'exécution d'une page Web. En effet, le navigateur repose sur les polices installées sur le système, mais cette pratique limite l'utilisation des polices spéciales qui, souvent, ne sont pas disponibles chez la plupart des clients. La solution est donc d'envoyer la police désirée au client pour s'en servir. C'est possible en CSS grâce à la règle @font-face.

```
@font-face{  
    font-family:Nom_de_la_police;  
    src:url("Chemin_de_la_police_sur_le_serveur");  
}
```

- Nom_de_la_police: désigne le nom que l'on veut attribuer à la police. Ce nom sera utilisé plus tard dans les propriétés font-family.
- Chemin_de_la_police_sur_le_serveur: désigne le chemin (absolu ou relatif) de la police (qui est disponible sur le serveur).

CSS : « Polices »

✓ **Propriétés de font :**

- **Police embarquée** : ou polices en ligne. Il s'agit d'une police qui est envoyée au client par le serveur pour s'en servir durant l'exécution d'une page Web. En effet, le navigateur repose sur les polices installées sur le système, mais cette pratique limite l'utilisation des polices spéciales qui, souvent, ne sont pas disponibles chez la plupart des clients. La solution est donc d'envoyer la police désirée au client pour s'en servir. C'est possible en CSS grâce à la règle @font-face.

```
@font-face{
    font-family:htower;
    src:url("fonts/htower.ttf");
}
h1{
    font-weight:normal;
    font-family:htower;
}
```

Police embarquée

CSS : « Polices »

✓ Propriétés de font :

- **Police embarquée** : ou polices en ligne. Il s'agit d'une police qui est envoyée au client par le serveur pour s'en servir durant l'exécution d'une page Web. En effet, le navigateur repose sur les polices installées sur le système, mais cette pratique limite l'utilisation des polices spéciales qui, souvent, ne sont pas disponibles chez la plupart des clients. La solution est donc d'envoyer la police désirée au client pour s'en servir. C'est possible en CSS grâce à la règle @font-face.
- TTF c'est le format le plus populaire, mais n'est pas supportée par tous les navigateurs. La solution consiste donc à convertir la police en plusieurs formats pris en charge par les navigateurs et qui sont: TTF, WOFF et SVG, EOT.

```
@font-face{
  font-family:"htower";
  src:url("htower.eot");
  src:url('htower.woff') format("woff"),
    url("htower.svg") format("svg"),
    url("htower.ttf") format("truetype");
}
```

CSS : « Styles de textes, marges, dimensions et listes »

✓ Propriétés de texte :

- **Text-align** : permet d'aligner le texte au sein de son conteneur. Les différentes valeurs que l'on peut lui attribuer ne vous sont pas étrangères:
 - Left : C'est la valeur par défaut, elle permet d'aligner le texte (ou d'autres objets comme les images) à gauche du conteneur.
 - Right : Pour placer le contenu à droite.
 - center: Sert à center le contenu.
 - Justify : Permet de justifier le contenu en l'étalant sur toute la longueur de la ligne. Les lignes (d'un paragraphe par exemple) finissent donc toutes à la même position.
- **Text-indent** : permet d'appliquer une indentation (alinéa) aux balises de type block (comme <section>, <aside>, <p>, <div>...). La première ligne de la balise qui reçoit le style marquera un retrait par rapport aux autres lignes.
- La valeur de text-indent s'exprime en pixel.

CSS : « Styles de textes, marges, dimensions et listes »

✓ Propriétés de texte :

- **Text-decoration** : permet de souligner, surligner ou barrer un texte.
 - underline: Le texte est souligné (comme le ferait la balise HTML <u>).
 - overline: Le texte est surligné. La ligne se trouvera donc en dessus du texte, mais c'est rarement utilisé.
 - line-through: Le texte est barré. La ligne passe à travers le texte (comme le ferait la balise HTML <s>).
 - none: Aucune décoration ne sera appliquée au texte (toutes les lignes seront retirées).
- **Letter-spacing** : permet d'appliquer un espacement entre les caractères d'un texte. Cet espacement peut être exprimé en pixel.

CSS : « Styles de textes, marges, dimensions et listes »

✓ Propriétés de texte :

▪ Exemple :

```
<h1>Cours de CSS</h1>
```

```
<h2>Résumé</h2>
```

```
<div>
```

CSS (Cascading Style Sheets) ou `<span>feuilles de style en cascade</span>` est un langage de style simple et puissant. Il permet de donner du look à votre page Web et la rendre ainsi présentable.

Si HTML sert à intégrer du contenu, le CSS quant à lui, applique du design à ce contenu. Il couvre tous les aspects visuelles d'une page Web, commençant de la simple coloration des objets et allant jusqu'aux et transformations complexes de ceux ci et le responsive design.

```
</div>
```

```
body{
  font-family:verdana, arial, sans-serif;
}
h1{
  text-align:center;
  letter-spacing:4px;
  color:#FFFFFFF;
  background-color:#000000;
}
h2{
  text-transform:uppercase;
}
div{
  text-indent:20px;
  text-align:justify;
  font-size:10pt;
}
span{
  text-decoration:underline;
}
```

Cours de CSS

RÉSUMÉ

CSS (Cascading Style Sheets) ou feuilles de style en cascade est un langage de style simple et puissant. Il permet de donner du look à votre page Web et la rendre ainsi présentable.

Si HTML sert à intégrer du contenu, le CSS quant à lui, applique du design à ce contenu. Il couvre tous les aspects visuelles d'une page Web, commençant de la simple coloration des objets et allant jusqu'aux et transformations complexes de ceux ci et le responsive design.

CSS : « Styles de textes, marges, dimensions et listes »

✓ Marge externe : famille marge :

- **margin** : exprime les marges externes. Sa valeur est exprimée en pixel.
- margin-left: marge externe gauche.
- margin-top: marge externe supérieure.
- margin-right: marge externe droite.
- margin-bottom: marge externe inférieure.
- Exemple :
- margin:10px => cela signifie que l'objet auquel on l'a appliqué aura une marge qui l'entoure et qui mesure 10px (en haut, en bas, à droite et à gauche).

```
img{  
    margin:10px;  
    margin-bottom:20px;  
}
```

CSS : « Styles de textes, marges, dimensions et listes »

✓ **Marge interne : famille padding :**

- **padding** : exprime les marges internes. Sa valeur est exprimée en pixel.
- padding-left: marge interne gauche.
- padding-top: marge interne supérieure.
- padding-right: marge interne droite.
- padding-bottom: marge interne inférieure.

CSS : « Styles de textes, marges, dimensions et listes »

✓ Largeur et hauteur :

- Width / Height
- Min-width / Min-height : largeur ou la hauteur minimale.
- Max-width / Max-height : largeur ou la hauteur maximale.

CSS : « Styles de textes, marges, dimensions et listes »

✓ Les listes :

- list-style-type : permet de définir le type de puce des éléments de la liste. Elle peut accepter les valeurs vues en HTML (disc, circle et square).
 - decimal : pour les nombres décimaux commençant de 1.
 - decimal-leading-zero : pour les nombres décimaux avec zéro initial commençant de 01.
 - upper-roman : pour les nombres romains majuscules commençant de I.
 - lower-roman : pour les nombres romains minuscules commençant de i.
 - lower-greek : pour la numération grecque.
 - lower-alpha : pour les lettres de l'alphabet minuscules.
 - upper-alpha : pour les lettres de l'alphabet majuscules.
- list-style-image : définir nos propres puces
- Syntaxe : `list-style-image:url("chemin_de_l_image")`

CSS : « Bordures »

- ✓ Une bordure est un cadre qui entoure des objets comme les images, cellules d'un tableau, conteneurs...
- **Border-color :** permet de définir la couleur de la bordure. Elle peut être définie par son nom (anglais), le code hexadécimal, le code décimal (rgb) ou le code décimal avec prise en charge de la transparence (rgba).
- **Border-width :** permet de définir l'épaisseur de la bordure. Elle est exprimée en pixel (l'unité la plus appropriée). Cependant on peut recourir à d'autres unités comme cm, mm, in...
- **Border-style :** permet de définir le style de la bordure :
 - solid: pour une bordure solide, c'est à dire, tracée sous forme de trait continu.
 - dotted: pour une bordure pointillée.
 - dashed: pour une bordure constituée de tirets.
 - double: la bordure est doublée (deux traits parallèles).
 - ridge: donne une bordure en relief.
 - inset: pour une bordure en relief vers l'intérieur.
 - outset: pour une bordure en relief vers l'extérieur.
 - none: aucune bordure n'est appliquée.
 - inherit: hérite le même style de bordure que l'élément parent (conteneur).

CSS : « Bordures »

- ✓ Une bordure est un cadre qui entoure des objets comme les images, cellules d'un tableau, conteneurs...
- **Border-style :** permet de définir le style de la bordure :
 - solid: pour une bordure solide, c'est à dire, tracée sous forme de trait continu.
 - dotted: pour une bordure pointillée.
 - dashed: pour une bordure constituée de tirets.
 - double: la bordure est doublée (deux traits parallèles).
 - ridge: donne une bordure en relief.
 - inset: pour une bordure en relief vers l'intérieur.
 - outset: pour une bordure en relief vers l'extérieur.
 - none: aucune bordure n'est appliquée.
 - inherit: hérite le même style de bordure que l'élément parent (conteneur).

```
border-top-style:solid;  
border-right-style:dotted;  
border-bottom-style:dashed;  
border-left-style:ridge;
```

```
border-style:solid dotted dashed ridge;
```

```
border:solid 1px #0000FF;
```

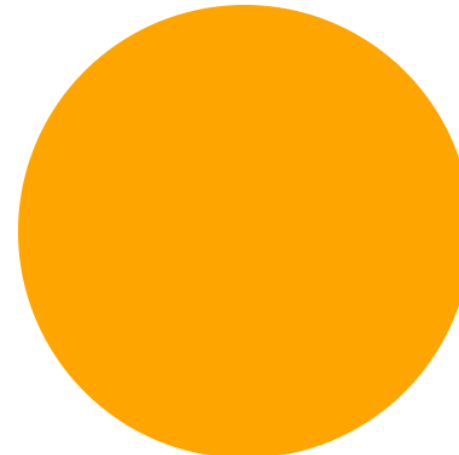
CSS : « Bordures »

- ✓ Une bordure est un cadre qui entoure des objets comme les images, cellules d'un tableau, conteneurs...
- **Border-radius :** applique un arrondi sur les coins de l'objet qui accueille la bordure. Radius = Rayon.

```
div{  
  width:300px;  
  height:300px;  
  background-color:orange;  
  border-radius:10px;  
}
```



```
div{  
  width:300px;  
  height:300px;  
  background-color:orange;  
  border-radius:150px;  
}
```



CSS : « Bordures »

- ✓ Une bordure est un cadre qui entoure des objets comme les images, cellules d'un tableau, conteneurs...
- **Border-radius :** applique un arrondi sur les coins de l'objet qui accueille la bordure.
- Peut appliquer des arrondis différents à chaque angle d'un objet

```
border-radius:Apx Bpx Cpx Dpx;
```

- A, B, C et D représentent respectivement les valeurs des arrondis appliquées aux angles : supérieur gauche, supérieur droit, inférieur droit et inférieure gauche. (sens horaire).

CSS : « Positions »

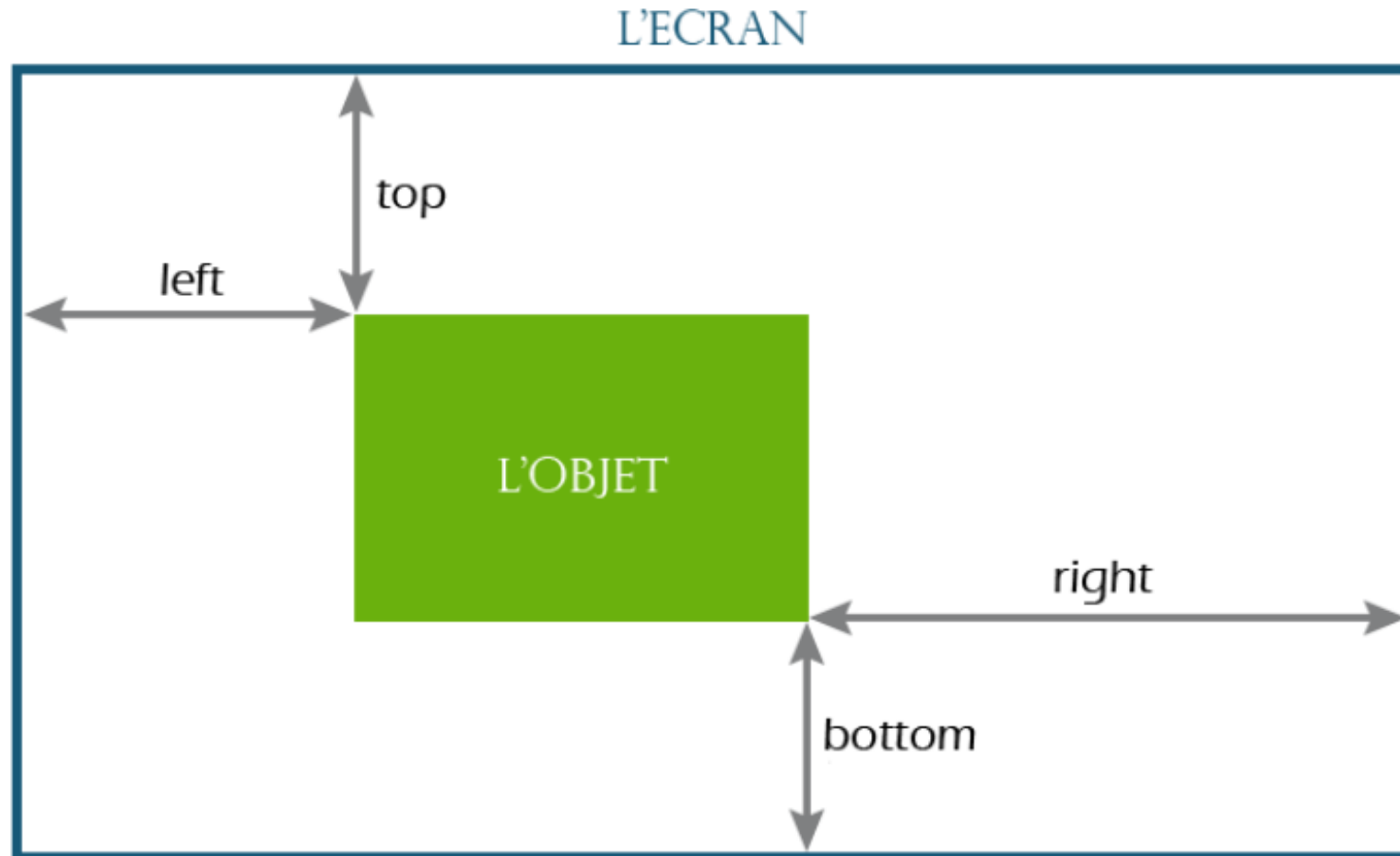
- **position** : permet de rendre l'ordre d'affichage des objets indépendant de l'ordre de leur déclaration en HTML.
 - static: Il s'agit de la valeur par défaut qui n'applique aucun positionnement particulier à l'objet qui est intégré normalement à l'emplacement décrit par le flux.
 - absolute: Cette valeur permet de rendre la position de l'objet absolue, ce qui donne la possibilité de survoler les autres objets. Il peut donc cacher un texte ou une image totalement ou partiellement selon l'endroit exacte où on l'a placé.
 - relative: Cette valeur permet de décaler l'affichage d'un objet par rapport à son emplacement naturel (celui décrit par HTML).
 - fixed: Cette valeur permet de fixer un élément sur la page même si celle-ci est défilée à l'aide des barres de défilement.
 - sticky: Il s'agit d'une position qui alterne entre static et fixed. Si on spécifie la propriété top à l'objet positionné en sticky, alors tant que sa position verticale est supérieure à la valeur spécifiée il est statique, et s'il passe en dessous de cette valeur alors il devient fixed.
- Pour pouvoir placer les objets là on l'on souhaite, utiliser d'autre propriétés CSS qui permettent de spécifier les coordonnées (déclarer avec précision l'emplacement de chaque objet auquel on a appliqué la propriété position).

CSS : « Positions »

- **position** : permet de rendre l'ordre d'affichage des objets indépendant de l'ordre de leur déclaration en HTML.
- Quand on parle de coordonnées, on pense à un repère. Le repère correspond à la fenêtre du navigateur.
- L'origine du repère est le coin haut à gauche du navigateur. Le sens positif horizontalement est vers la droite et verticalement vers le bas. Pour spécifier le x et le y on fait appel aux propriétés CSS left, top, right et bottom.
 - left: exprime le nombre de pixels qui séparent l'extrémité gauche de l'objet du côté gauche du navigateur.
 - top: exprime le nombre de pixels qui séparent l'extrémité supérieure de l'objet du haut du navigateur.
 - right: exprime le nombre de pixels qui séparent l'extrémité droite de l'objet du côté droit du navigateur.
 - bottom: exprime le nombre de pixels qui séparent l'extrémité inférieure de l'objet du bas du navigateur.

CSS : « Positions »

- **position** : permet de rendre l'ordre d'affichage des objets indépendant de l'ordre de leur déclaration en HTML.



CSS : « Positions »

- **position** : permet de rendre l'ordre d'affichage des objets indépendant de l'ordre de leur déclaration en HTML.
- **Inconvénients** :



Consultation Stratégique

Des conseils personnalisés pour booster votre croissance.

Développement Web

Des sites web modernes et performants adaptés à tous les appareils.



Stratégique

Des conseils personnalisés pour booster votre croissance.



CSS : « Positions »

- **Z-index :** permet de spécifier la valeur Z sur l'axe (virtuel) sortant de l'écran. Il a comme valeur un nombre entier. Plus la valeur de z-index est grande plus l'objet est placé au premier plan.



nt Web

ernes et performants adaptés à tous les appareils.

Styles

Computed

Layout

Filter

```
element.style {  
  position: absolute;  
}
```



Developpement Web

Des sites web modernes et performants adaptés à tous les appareils.

Styles

Computed

Layo

Filter

```
element.style {  
  z-index: 2;  
}
```

CSS : « Transformation des éléments »

- Les éléments de type **inline** occupent juste assez d'espace pour qu'ils s'affichent correctement. Par exemple la balise `<span>` est de type inline. Elle n'engendre pas de retour à la ligne automatique car elle se contente de l'espace nécessaire pour que ce qu'elle contient (texte ou autre objet) s'affiche convenablement. C'est le cas pour les balises `<b>`, `<i>`, `<img>`...
- Les balises de type **block** constituent un "bloc". Comme la balise `<div>` elle occupe toute la largeur de la page même si son contenu n'est pas aussi large. C'est ce qui explique le retour à la ligne automatique qui survient avant et après ce type de balises. Les balises `<header>`, `<footer>`, `<p>` et bien d'autres font aussi partie de cette famille.
- **Exemple :**

```
<div>Balise DIV</div>
<span>Balise SPAN</span>
<h1>Balise H1</h1>
<font>Balise FONT</font>
```

Balise DIV

Balise SPAN

Balise H1

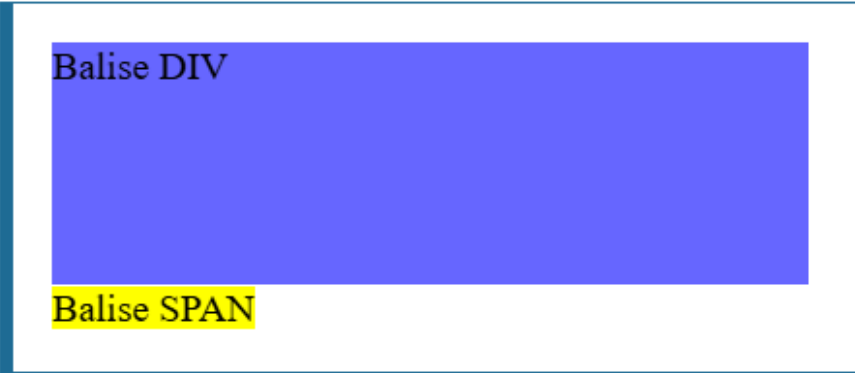
Balise FONT

```
div{
    background-color:#6666FF;
}
span{
    background-color:#FFFF00;
}
h1{
    background-color:#FF8800;
}
font{
    background-color:#AAAAAA;
}
```

CSS : « Transformation des éléments »

- Seules les balises de type **block** sont redimensionnables. Les balises **inline** ignorent les styles de redimensionnement à l'exception des images et les champs de formulaire.
- **Exemple :**

```
<div>Balise DIV</div>  
<span>Balise SPAN</span>
```



Balise DIV

Balise SPAN

```
div{  
    background-color:#6666FF;  
    width:50%;  
    height:100px;  
}  
span{  
    background-color:#FFFF00;  
    width:50%;  
    height:100px;  
}
```

CSS : « Transformation des éléments »

- **display :** Le CSS a prévu des styles qui transforment les balises, c'est à dire qu'on peut rendre la balise `<span>` de type block et la balise `<div>` de type inline et cela s'applique sur toutes les balises HTML. La propriété qui rend cet exploit possible s'appelle **display**.
 - inline: permet de transformer une balise HTML en type inline.
 - block: permet de transformer une balise HTML en type block.
 - none: permet de faire disparaître l'élément HTML. L'élément aura l'air comme s'il n'a pas été déclaré dans le code HTML.
 - inline-block: cette valeur a été introduite en CSS3. Elle permet de réunir les principales caractéristiques de chacune des types inline et block. Si on applique la propriété `display:inline-block` à un élément, il n'engendre pas de retour automatique à la ligne (comme les inline), et peut être redimensionné (comme les block).
 - flex: permet de créer un conteneur flexible, facilitant l'alignement et la disposition des éléments enfants (appelés flex items) de manière responsive et adaptable.

CSS : « Transformation des éléments »

- **display :** Le CSS a prévu des styles qui transforment les balises, c'est à dire qu'on peut rendre la balise `<span>` de type block et la balise `<div>` de type inline et cela s'applique sur toutes les balises HTML. La propriété qui rend cet exploit possible s'appelle **display**.
 - inline: permet de transformer une balise HTML en type inline.
 - block: permet de transformer une balise HTML en type block.

```
<div>Balise DIV</div>  
<span>Balise SPAN</span>
```



Balise DIV
Balise SPAN

```
div{  
    background-color:#6666FF;  
    width:50%;  
    height:100px;  
    display:inline;  
}  
span{  
    background-color:#FFFF00;  
    width:50%;  
    height:100px;  
    display:block;  
}
```

CSS : « Transformation des éléments »

- float : permet de faire sortir l'élément du flux normal).

```
<h1>CSS c'est quoi?</h1>
```

CSS désigne Cascading Style Sheets (pour Feuilles de style en cascade). Il s'agit d'un langage de style dont la syntaxe est extrêmement simple mais son rendement est remarquable. En effet, le CSS s'intéresse à la mise en forme du contenu intégré avec du HTML.

Donc, si vous voulez changer la couleur de votre arrière plan, la police de vos textes ou l'alignement et les marges de vos objets et bien réussir d'autres prouesses, CSS est là pour vous servir.

```
<br />
```

```

```

CSS c'est quoi?

CSS désigne Cascading Style Sheets (pour Feuilles de style en cascade). Il s'agit d'un langage de style dont la syntaxe est extrêmement simple mais son rendement est remarquable. En effet, le CSS s'intéresse à la mise en forme du contenu intégré avec du HTML.

Donc, si vous voulez changer la couleur de votre arrière plan, la police de vos textes ou l'alignement et les marges de vos objets et bien réussir d'autres prouesses, CSS est là pour vous servir.



CSS : « Transformation des éléments »

- **float** : permet de faire sortir l'élément du flux normal).

```
img{  
  float:left;  
}
```



CSS c'est quoi?

CSS désigne Cascading Style Sheets (pour Feuilles de style en cascade). Il s'agit d'un langage de style dont la syntaxe est extrêmement simple mais son rendement est remarquable. En effet, le CSS s'intéresse à la mise en forme du contenu intégré avec du HTML.

Donc, si vous voulez changer la couleur de votre arrière plan, la police de vos textes ou l'alignement et les marges de vos objets et bien réussir d'autres prouesses, CSS est là pour vous servir.

CSS : « Transformation des éléments »

- **float** : permet de faire sortir l'élément du flux normal).
- La propriété float peut accueillir les valeurs suivantes:
 - left: l'objet flotte à gauche et les autres éléments se répartiront autour.
 - right: l'objet flotte à droite et les autres éléments se répartiront autour.
 - none: aucun flottement n'est appliqué.
- **Propriété clear** : sert à annuler l'effet d'un élément flottant sur l'élément sur laquelle elle est appliquée.
 - left: cesse l'effet d'un flottant à gauche (float:left) sur l'élément courant.
 - right: cesse l'effet d'un flottant à droite(float:right) sur l'élément courant.
 - both: cesse l'effet d'un flottant à gauche ou à droite (float:left ou float:right) sur l'élément courant.
- **Propriété vertical-align** : définit l'alignement vertical d'un élément de type inline ou d'une cellule d'un tableau.
 - baseline: C'est la valeur par défaut, elle aligne la base de l'élément avec la base du texte contenu dans son parent.
 - top: Aligne le haut de l'élément avec le haut du texte contenu dans son parent. Dans le cas d'un tableau, il aligne l'élément en haut de la cellule.
 - bottom: Aligne le bas de l'élément avec le bas du texte contenu dans son parent. Dans le cas d'un tableau, il aligne l'élément en bas de la cellule.
 - middle: Aligne le milieu de l'élément avec le milieu du texte (considéré en minuscules) contenu dans son parent. Dans le cas d'un tableau, il centre verticalement l'élément dans la cellule.

CSS : « Masquage des éléments »

- **visibility** : permet de masquer ou d'afficher un élément.
 - visible: C'est le comportement normal, l'élément est donc visible normalement.
 - hidden: l'élément est masqué mais son espace reste toujours visible.

```
<div>Avant l'image</div>  
  
<div>Après l'image</div>
```

```
img{  
    visibility:hidden;  
}
```

Avant l'image

Après l'image

CSS : « Opacité »

- **opacity** : permet de rendre un élément transparent, opaque ou translucide.
- Sa valeur est comprise entre 0 et 1 avec un pas de 0.01. 0 signifie complètement transparente, 1 veut dire complètement opaque et entre 0 et 1 ce sont les 99 niveaux de transparence possibles.
- Internet Explorer version inférieure à 9 reconnaît une autre syntaxe : `filter:alpha(opacity=x)` où x est la valeur de l'opacité comprise entre 0 et 100 (0 pour transparent et 100 pour opaque).

```

```

```
img{  
  opacity:0.3;  
  filter:alpha(opacity=30);  
}
```



CSS : « Gestion de débordement »

<div>

CSS désigne Cascading Style Sheets (pour Feuilles de style en cascade). Il s'agit d'un langage de style dont la syntaxe est extrêmement simple mais son rendement est remarquable. En effet, le CSS s'intéresse à la mise en forme du contenu intégré avec du HTML.

Donc, si vous voulez changer la couleur de votre arrière plan, la police de vos textes ou l'alignement et les marges de vos objets et bien réussir d'autres prouesses, CSS est là pour vous servir.</div>

CSS désigne Cascading Style Sheets (pour Feuilles de style en cascade). Il s'agit de style dont la syntaxe est extrêmement simple mais son rendement est remarquable. En effet, le CSS s'intéresse à la mise en forme du contenu intégré avec du HTML.

Donc, si vous voulez changer la couleur de votre arrière plan, la police de vos textes ou l'alignement et les marges de vos objets et bien réussir d'autres prouesses, CSS est là pour vous servir.

```
div{
  height:60px;
  border:solid 1px #AAAAAA;
  padding:10px;
}
```

CSS : « Gestion de débordement »

- **overflow** : permet de gérer le débordement sur un élément (de type conteneur).
- visible: C'est la valeur par défaut. Le contenu qui dépasse l'élément est visible normalement.
- hidden: Le contenu qui dépasse sera masqué.
- scroll: Une barre de défilement apparaît qu'il y ait débordement ou non, mais elle ne sera opérationnelle que si le contenu dépasse de l'élément. Notez que l'imprimante peut imprimer tout le contenu, même celui qui dépasse.
- auto: Une barre de défilement apparaît s'il y a un débordement.

```
div{  
  height:60px;  
  border:solid 1px #AAAAAA;  
  padding:10px;  
  overflow:scroll;  
}
```

CSS désigne Cascading Style Sheets (pour Feuilles de style en cascade). Il s'agit d'un langage de style dont la syntaxe est extrêmement simple mais son rendement est remarquable. En effet, le CSS s'intéresse à la mise en forme du contenu intégré avec du

CSS : « Sélcteurs spéciaux »

- **L'attribut class : Groupement d'éléments**

- Cet attribut peut avoir comme valeur un nom (au choix) et peut être appliqué à toutes les balises HTML déclarées entre <body> et </body>.
- On peut appliquer le même nom de classe à plusieurs éléments similaires ou non.
- Cet ensemble d'éléments constitue un groupement d'éléments.

```
<h1 class="titre"></h1>
<nav class="menu"></nav>
<div class="contenu"></div>
<article>
  <div class="meteo"></div>
  <div class="rss"></div>
</article>
```

```
.contenu{
  /* Style de la première DIV */
}
.meteo{
  /* Style de la deuxième DIV */
}
.rss{
  /* Style de la troisième DIV */
}
```

CSS : « Sélécteurs spéciaux »

- L'attribut id : identifiant d'un élément
- ne doit figurer qu'une seule fois dans le même document.

```
<img id="logo" />  
<div id="slogan"></div>
```

```
#logo{  
    /* Style de l'image affichant le logo */  
}  
#slogan{  
    /* Style de la DIV qui contient le slogan */  
}
```

CSS : « Sélecteurs spéciaux »

- **Cibler tous les éléments** : cible tous les éléments d'un document HTML, on utilise le symbole astérisque (*).

```
*{  
  color:#888888;  
  font-size:10pt;  
  font-family:verdana, arial, sans-serif;  
}
```

- **Cibler tous les éléments ayant un attribut particulier** : cible toutes les balises sur lesquelles on a déclaré un attribut particulier (quel que soit sa valeur).
- On fait appel au sélecteur de balise suivi de crochets qui renferment le nom de l'attribut souhaité.

```
img[title]{  
  opacity:0.7;  
  filter:alpha(opacity=70);  
}
```

CSS : « Sélecteurs spéciaux »

- **Cibler tous les éléments** : cible tous les éléments d'un document HTML, on utilise le symbole astérisque (*).

```
*{  
  color:#888888;  
  font-size:10pt;  
  font-family:verdana, arial, sans-serif;  
}
```

- **Cibler tous les éléments ayant un attribut particulier** : cible toutes les balises sur lesquelles on a déclaré un attribut particulier (quel que soit sa valeur).
- On fait appel au sélecteur de balise suivi de crochets qui renferment le nom de l'attribut souhaité.

```
img[title]{  
  opacity:0.7;  
  filter:alpha(opacity=70);  
}
```

CSS : « Sélecteurs spéciaux »

- **Cibler un élément contenu dans un autre :** cible un élément contenu dans un autre, on déclare le sélecteur du premier élément suivi d'un espace suivi du sélecteur du deuxième élément. Le sélecteur peut être n'importe lequel parmi ceux vu précédemment (balise, classe, id...).

```
div img{  
    margin:10px;  
}
```

- **Cibler un élément enfant direct d'un parent :** Si on veut désigner un élément enfant appartenant directement à un parent spécifié alors on utilise le symbole >.

```
div>img{  
    margin:10px;  
}
```

CSS : « Sélcteurs spéciaux »

- **Cibler plusieurs éléments à la fois :** cible plusieurs éléments à la fois séparés par une virgule.

```
header, footer, nav, section{  
    margin:0;  
    border:none;  
}
```

- **Cibler un élément précédé immédiatement par un autre :** spécifier un élément HTML qui suit directement un élément particulier en utilisant le caractère +.

```
nav+h1{  
    margin:auto;  
}
```

- Le style sera appliqué à la balise <h1> qui vient directement après la balise <nav>. C'est à dire qu'on a déjà fermé la balise <nav> puis on a ouvert la balise <h1> juste après.

CSS : « Sélecteurs spéciaux »

- Ordre d'application des styles :
- Les propriétés qui ne sont pas répétées d'un style à un autre sont combinées et appliquées toutes à l'élément
- Les règles qui se répètent s'annulent les unes par les autres : celles qui sont définies en dernier seront appliquées.

```
▼ <body>
```

```
 == $0
```

```
</body>
```

```
</html>
```

```
element.style {  
  float: left;  
  width: 150px;  
  margin-right: 10px;  
  width: 100px;
```

CSS : « Pseudo-classes »

- **Pseudo-classes :**
 - Une pseudo-classe est un mot clé préfixé par deux points (:) qui s'ajoute à un sélecteur CSS pour appliquer un style à un élément dans un cas particulier.
 - Utilisation : les liens hypertextes qui changent de décor selon si ils sont nouvellement affichés, déjà visités ou survolés..., les images, les conteneurs, les listes...
- **Pseudo-classe :link :** désigne un lien hypertexte dont la page cible (celle spécifiée dans son attribut href) n'a pas encore été visitée. Elle peut aider les internautes à distinguer les liens qu'ils n'ont pas encore exploré.
- **Pseudo-classe :visited :** à l'inverse de link, la pseudo-classe :visited désigne un lien hypertexte dont la page cible a déjà été visitée. C'est un genre d'historique en quelque sorte.
- **Pseudo-classe :active :** désigne un lien hypertexte sélectionné. Il s'agit de l'instant où le visiteur clique sur le lien.
- **Pseudo-classe :hover :** désigne un objet survolé par le curseur de la souris, il peut être un hyperlien ou n'importe quel autre objet.
- **Pseudo-classe :focus :** désigne un élément activé ou sélectionné suite à un clic ou une tabulation. C'est souvent utilisé sur les champs de formulaire.

CSS : « Pseudo-classes »

- **Pseudo-classe :first-child** : désigne le premier élément enfant. Supposons que nous avons déclaré le sélecteur suivant `span:first-child`. Le navigateur appliquera le style associé à toutes les balises `<span>` qui figurent en tant que premier élément enfant de n'importe quelle balise.

```
<div>
  <span>Bonjour</span>
  à
  <span>tous</span>
</div>
```

- Seule la première balise `<span>` s'attribuerait le style décrit dans le sélecteur.
- **Pseudo-classe :nth-child(n)** : permet d'accéder à un élément enfant de n'importe quel rang et pas que le premier. Le paramètre `n` mis entre les parenthèses désigne le rang de l'élément à styler. Il s'agit d'un indice numérique qui commence de 1 (1 étant le premier élément, 2 le deuxième et ainsi de suite...).
- Pour l'exemple HTML précédent, le sélecteur `div>span:nth-child(2)` fait référence à la deuxième balise `<span>`.

CSS : « Pseudo-elements »

- **Pseudo-éléments** : accèdent à certaines parties de l'élément pour les styler.
- **Pseudo-élément ::first-letter** : désigne la première lettre de l'élément auquel il est associé. Il permet de donner un style particulier à la première lettre d'un élément.

```
<div>  
  La première lettre de ce texte est différente!  
</div>
```

```
div::first-letter{  
  font-size:24pt;  
  color:orange;  
}
```

La première lettre de ce texte est différente!

CSS : « Pseudo-elements »

- Pseudo-éléments : accèdent à certaines parties de l'élément pour les styler.
- Pseudo-élément ::first-line : désigne la première ligne de l'élément.

```
<p class="paragraphe">  
Tout comme les pseudo-classes, les pseudo-éléments sont préfixés  
par deux points (:) et ajoutés au sélecteur. Si les pseudo-classe  
décrivent un état d'un élément (comme un lien hypertexte  
survolé), les pseudo-éléments eux accèdent à certaines parties de  
l'élément pour les styler.  
</p>
```

```
.paragraphe::first-line{  
    color:blue;  
}
```

Tout comme les pseudo-classes, les pseudo-éléments sont préfixés par deux points (:) et ajoutés au sélecteur. Si les pseudo-classe décrivent un état d'un élément (comme un lien hypertexte survolé), les pseudo-éléments eux accèdent à certaines parties de l'élément pour les styler.

CSS : « Pseudo-elements »

- **Pseudo-éléments :** accèdent à certaines parties de l'élément pour les styler.
- **Pseudo-éléments ::before :** permet de créer un pseudo-élément au début de l'élément sur lequel il est déclaré. Exemple : intégrer du texte à un élément grâce au CSS...
- Cette pratique devrait rester limitée en utilisation. C'est le HTML qui devrait créer le contenu et le CSS ne fait que le styler.
- **Pseudo-éléments ::after :** crée un pseudo-élément à la fin de l'élément sur lequel il est déclaré.
- Les pseudo-éléments ::before et ::after font appel à la propriété content qui ajoute du contenu (par défaut de type inline).

```
<div>
  Contenu de la DIV
</div>
```

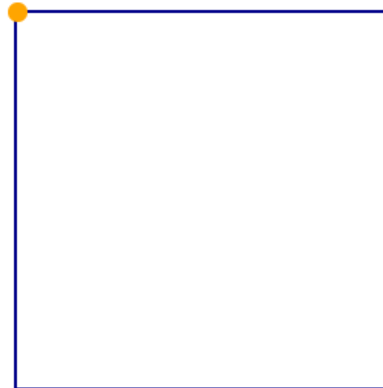
```
div::before{
  content:"Avant";
  color:red;
}
div::after{
  content:"Après";
  color:blue;
}
```

Avant Contenu de la DIV Après

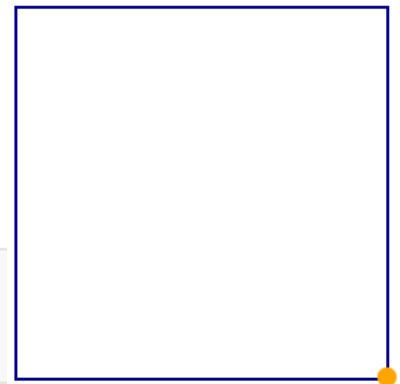
CSS : « Transformations »

- A partir de la version 3 de CSS il est devenu possible de transformer les objets intégrés en 2D (et même en 3D) sans vraiment modifier leurs propriétés (largeur, hauteur...).
- Les transformations d'objets sont appliquées par rapport à son point d'origine. Ce point est par défaut situé au centre l'objet.
- **Propriété transform-origin :** permet de modifier le point d'origine d'un objet HTML. Il accepte comme valeur le X et le Y de l'origine souhaité séparés par un espace. Ces deux informations peuvent être spécifiées en pourcentage, en pixel, ou par les mots clé left, right, top, bottom et center.

```
transform-origin:0 0;
```



```
transform-origin:100% 100%;
```



CSS : « Transformations »

- **Fonctions de transformations** : **transform**: fonction1() fonction2() fonction3()...
- **Fonction translate()** : permet la translation de l'objet HTML d'un emplacement initial (où il est normalement défini) à un autre emplacement. Les coordonnées du nouvel emplacement sont renseignées entre les parenthèses de translate() et ils sont exprimés en pixel. L'origine des coordonnées est le point haut à gauche.

```
div{  
  width:100px;  
  height:100px;  
  background-color:orange;  
  transform:translate(100px,30px);  
}
```



- La forme pointillée représente l'objet sans transformation.
- Le premier paramètre de la fonction translate() représente la translation horizontale, et le deuxième la translation verticale.

CSS : « Transformations »

- **Fonction scale()** : permet de redimensionner l'objet HTML (avec son contenu). Les parenthèses de la fonction peuvent accueillir une seule valeur ou deux valeurs séparées par une virgule. Ces valeurs désignent le facteur d'agrandissement. Ils n'ont pas d'unité et peuvent contenir de virgule. S'ils sont supérieurs à 1 alors l'objet est agrandi, sinon alors il est rétréci.



```
div{  
  width:100px;  
  height:100px;  
  background-color:orange;  
  transform-origin:0 0;  
  transform:scale(2,0.8);  
}
```



- Il existe également les fonctions `scaleX()` et `scaleY()` qui appliquent respectivement un facteur d'agrandissement uniquement sur la largeur ou la hauteur.

CSS : « Transformations »

- **Fonctions skewX() et skewY()** : permettent d'incliner un objet respectivement horizontalement et verticalement par rapport à son origine. La valeur renseignée dans les parenthèses désigne l'angle d'inclinaison et elle est exprimée en degré (deg).

```
div{  
  width:100px;  
  height:100px;  
  background-color:orange;  
  transform-origin:0 0;  
  transform:skewX(30deg);  
}
```



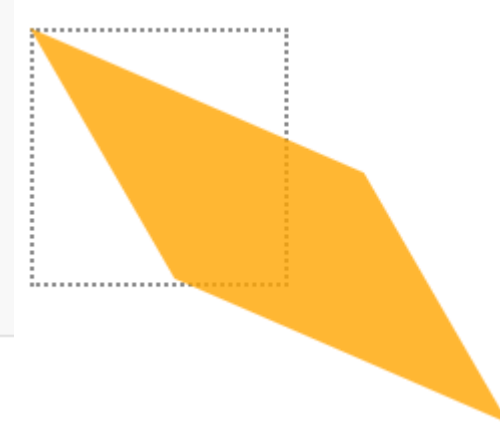
```
div{  
  width:100px;  
  height:100px;  
  background-color:orange;  
  transform-origin:0 0;  
  transform:skewY(30deg);  
}
```



CSS : « Transformations »

- **Fonctions skewX() et skewY()** : permettent d'incliner un objet respectivement horizontalement et verticalement par rapport à son origine. La valeur renseignée dans les parenthèses désigne l'angle d'inclinaison et elle est exprimée en degré (deg).

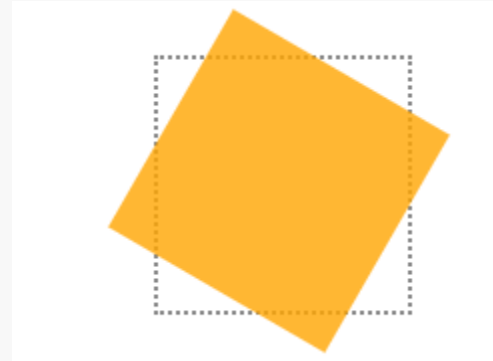
```
div{  
  width:100px;  
  height:100px;  
  background-color:orange;  
  transform-origin:0 0;  
  transform:skewX(30deg) skewY(30deg);  
}
```



CSS : « Transformations »

- **rotate()** : applique une rotation à l'objet autour de l'axe traversant son origine. Les parenthèses contiennent l'angle de la rotation exprimée en degré.
- rotate() ou rotateZ(): la rotation est appliquée par rapport à l'axe virtuel perpendiculaire à l'écran (axe des Z de z-index) qui traverse le point d'origine de l'objet. La rotation s'applique donc sur le même plan que l'écran.

```
div{  
  width:100px;  
  height:100px;  
  background-color:orange;  
  transform-origin:50% 50%;  
  transform:rotate(30deg);  
}
```



CSS : « Transformations »

- **rotate()** : applique une rotation à l'objet autour de l'axe traversant son origine. Les parenthèses contiennent l'angle de la rotation exprimée en degré.
- rotateX(): la rotation est appliquée par rapport à l'axe horizontale traversant l'origine de l'objet (l'axe des X). La rotation aura donc un effet 3D
- rotateY(): la rotation est appliquée par rapport à l'axe vertical traversant l'origine de l'objet (l'axe des Y). La rotation aura aussi un effet 3D

```
div{  
  width:100px;  
  height:100px;  
  background-color:orange;  
  transform-origin:50% 50%;  
  transform:rotateX(50deg);  
}
```



CSS : « Transformations »

- **perspective()** : permet de prendre en compte les changements des dimensions en fonction de la profondeur (ou perspectives).
- La fonction perspective() reçoit une valeur exprimée en pixel qui désigne la distance (virtuelle) qui sépare l'observateur de l'axe de rotation (qui traverse l'origine de l'objet). Plus cette distance est petite, plus l'effet de perspective est accentué et inversement.

```
div{  
  width:100px;  
  height:100px;  
  background-color:orange;  
  transform-origin:50% 50%;  
  transform:perspective(150px) rotateX(50deg);  
}
```



```
div{  
  width:100px;  
  height:100px;  
  background-color:orange;  
  transform-origin:50% 50%;  
  transform:perspective(150px) rotateY(50deg);  
}
```



CSS : « Transitions »

- Permettent le passage d'un objet d'un état à un autre d'une manière progressive créant ainsi une animation.
- **transition-property** : permet de désigner quels sont les propriétés qui devront subir la transition et quels sont celles qui ne s'animeront pas. Il s'agit de la propriété transition-property.
- La propriété transition-property accepte comme valeur le nom (ou la famille) de styles sur lesquelles la transition sera appliquée.

```
div{
  border:solid 1px blue;
  background-color:orange;
}
div:hover{
  border:dotted 1px green;
  background-color:yellow;
  transition-property:background;
}
```

- En passant de l'état initial à l'état survolé, la bordure et l'arrière-plan de la balise <div> changeront, mais seule l'arrière-plan subira la transition alors que la bordure changera instantanément.
- transition-property:all => toutes les propriétés auraient subi la transition.

CSS : « Transitions »

- Permettent le passage d'un objet d'un état à un autre d'une manière progressive créant ainsi une animation.
- **transition-duration :** permet de spécifier la durée de la transition (la durée que mettra l'animation pour aller de l'état 1 à l'état 2). Elle est exprimée en secondes.

```
div{
  border:solid 1px blue;
  background-color:orange;
}
div:hover{
  border:dotted 1px green;
  background-color:yellow;
  transition-property:background;
  transition-duration:0.5s;
}
```

CSS : « Transitions »

- Permettent le passage d'un objet d'un état à un autre d'une manière progressive créant ainsi une animation.
- **transition-timing-function** : permet de définir la cadence avec laquelle la transition sera faite. Elle permet de changer la vitesse de la transition au long de sa durée (on parle de la courbe d'accélération).
- ease: le mouvement est rapide au début puis ralentit petit à petit jusqu'à l'arrêt complet.
- linear: le mouvement garde la même vitesse du début à la fin de la transition.
- ease-in: le mouvement commence lentement puis gagne en vitesse avec le temps.
- ease-out: le mouvement est rapide au début puis ralentit à la fin (le ralenti avec ease-out est moins senti qu'avec ease).
- ease-in-out: le mouvement commence lentement et fini lentement, mais il est plus rapide au milieu de la transition.

```
div{
  border:solid 1px blue;
  background-color:orange;
}
div:hover{
  border:dotted 1px green;
  background-color:yellow;
  transition:all 0.5s ease;
}
```

CSS : « Transitions »

- Permettent le passage d'un objet d'un état à un autre d'une manière progressive créant ainsi une animation.
- **transition-delay** : permet de retarder le début de la transition lorsque l'événement qui la déclenche est détecté.

```
div{
  border:solid 1px blue;
  background-color:orange;
}
div:hover{
  border:dotted 1px green;
  background-color:yellow;
  transition-property:all;
  transition-duration:0.5s;
  transition-timing-function:ease;
  transition-delay:1s;
}
```

CSS : « Animations »

- Consistent à un ensemble de propriétés qui permettent de décrire l'évolution d'un objet HTML dans le temps par le changement de ses propriété CSS.
- Les transitions permettent d'appliquer une animation entre deux états d'un objet VS les animations permettent de définir à tout instant l'attitude de l'objet.
- Leur lancement ne demande pas forcément l'intervention de l'utilisateur et peuvent donc se lancer automatiquement et se répéter autant de fois que l'on souhaite.
- **Appliquer une animation à un objet: animation-name :** permet d'identifier la suite d'animations à lui appliquer parmi d'autres (s'il en existe sur la page courante).
- Cette propriété (animation-name) accepte comme valeur un mot de votre choix.
- Si des animations différentes sont présentes sur la même page, alors il faut veiller à ce que la valeur de animation-name soit unique pour éviter tout conflit.

```
#objet{  
    width:100px;  
    height:100px;  
    background-color:#E80;  
    animation-name:monanimation;  
}
```

CSS : « Animations »

- Définir la séquence d'animations à associer à un objet: @keyframes : contient les styles de chaque cas (une sorte d'image clé pour les animations interpolées).
- Les styles-clés sont spécifiés avec des valeurs en pourcentage. Par exemple, si l'animation dure 2 secondes alors 0% signifie l'instant 0s, 50% signifie l'instant 1s et 100% signifie l'instant 2s.
- Chaque valeur en pourcentage contient l'ensemble des styles-clés qui seront appliqués à l'instant approprié. Le changement de ces styles (entre un instant et l'instant suivant) se fait d'une manière progressive ce qui crée l'animation voulue.

```
<style>
  #objet{
    width:100px;
    height:100px;
    background-color:#E80;
    animation-name:monanimation;
  }
  @keyframes monanimation{
    0%{
      width:100px;
      height:100px;
      background-color:#E80;
    }
    50%{
      width:100px;
      height:100px;
      background-color:#0A0;
      border-radius:100px;
      transform:rotate(90deg);
    }
    100%{
      width:100px;
      height:100px;
      background-color:#E80;
      transform:rotate(180deg);
    }
  }
</style>
```

CSS : « Animations »

- **Durée de l'animation: animation-duration :** Il s'agit de la durée totale de l'animation entre l'instant 0% et l'instant 100%.
- **Rythme entre séquences de l'animation: animation-timing-function :** permet de changer la vitesse de l'animation entre deux séquences voisines (on parle de la courbe d'accélération).
- **Délai avant démarrage de l'animation: animation-delay :** définit la durée d'attente avant de lancer l'animation.
- **Nombre de répétitions de l'animation: animation-iteration-count :** La propriété animation-iteration-count définit le nombre de fois que l'animation sera jouée. Par défaut elle vaut 1 et l'animation se joue donc une seule fois puis s'arrête. Si on veut que l'animation tourne à l'infinie alors on applique la valeur infinite.
- **Sens de l'animation: animation-direction :** permet de définir le sens du mouvement.
 - normal: (valeur par défaut) l'animation sera jouée dans le sens normal de l'instant 0% à 100%
 - reverse: l'animation sera jouée dans le sens inverse, donc de 100% à 0%
 - alternate: l'animation sera jouée dans les deux sens, de 0% à 100% puis de 100% à 0% de manière consécutive.

CSS : « Animations »

- Exemple :

```
<style>
  #objet{
    width:100px;
    height:100px;
    background-color:#E80;
    animation-name:monanimation;
    animation-iteration-count:infinite;
    animation-duration:2s;
    animation-timing-function:ease;
    animation-direction:normal;
  }
  @keyframes monanimation{
    0%{
      width:100px;
      height:100px;
      transform:translate(0,0) rotate(0);
      background-color:#E80;
    }
    25%{
      width:100px;
      height:100px;
      transform:translate(100%,0) rotate(-90deg);
      background-color:#0A0;
    }
  }
```

```
    50%{
      width:100px;
      height:100px;
      transform:translate(100%,100%) rotate(-180deg);
      background-color:#F0F;
    }
    75%{
      width:100px;
      height:100px;
      transform:translate(0,100%) rotate(-270deg);
      background-color:#00A;
    }
    100%{
      width:100px;
      height:100px;
      transform:translate(0,0) rotate(-360deg);
      background-color:#E80;
    }
  }
</style>
```

CSS : « Media Queries (Responsive Design) »

- Les Media Queries (ou requêtes média) est une technique (apparue avec les spécifications CSS3) qui permet de définir des conditions pour appliquer des styles CSS. Ces conditions ont un rapport avec la nature et les propriétés du périphérique de sortie (écran, smartphone, imprimante...).
- Les Média Queries permettent de réussir ce qui est communément connu sous le nom du Responsive Design qui consiste à adapter dynamiquement de design à l'aide du CSS.
- **Attribut media** : déclaré sur la balise <link> pour indiquer au navigateur dans quel cas il faut appliquer la feuille de style associée.
- screen: signifie les écrans.
- print: pour les imprimantes.
- handheld: désigne les appareils mobiles ou écrans de petite taille.
- projection: signifie les projecteurs.
- tv: pour les téléviseurs.
- tty: signifie terminal (on peut visualiser une page Web sur un terminal comme l'invite de commande).
- all: qui désigne tous les périphériques.

```
<link rel="stylesheet" type="text/css" href="ecran.css"
media="screen" />
<link rel="stylesheet" type="text/css" href="imprimante.css"
media="print" />
```

CSS : « Media Queries (Responsive Design) »

- **Règle @media :** Au lieu de créer plusieurs feuilles de styles, chacune pour un type de périphérique, on peut créer une seule qui renferme tous les styles groupés par les conditions Media Queries commençant par @media.

```
@media screen{
  h1{
    font-size:24pt;
    font-weight:normal;
  }
  nav{
    display:inline-block;
  }
}
@media print{
  h1{
    font-size:18pt;
    font-weight:bold;
  }
  nav{
    display:none;
  }
}
```

CSS : « Media Queries (Responsive Design) »

- Règle @media :
- Opérateurs logiques :
 - and: qui signifie un ET logique. Par exemple, la règle A and B est vraie si A est vrai et B est vrai aussi.
 - only: qui signifie UNIQUEMENT. Par exemple, la règle only A est vraie si A tout seul est vrai.
 - not: qui signifie NON. Par exemple, la règle not A est vraie si A est fausse.

```
@media screen and (min-width:1280px){  
    .pub{  
        display:inline-block;  
    }  
}
```

CSS : « Media Queries (Responsive Design) »

- Règle @media :
- Critères :
- Placés entre des parenthèses pour constituer un élément de la condition à vérifier.
- width: signifie la largeur de la fenêtre d'affichage (est pas celle de l'écran). On peut préfixer ce critère par min- (pour min-width) ou max- (pour max-width) pour désigner respectivement la largeur minimale et la largeur maximale.
- height: signifie la hauteur de la fenêtre d'affichage. On peut préfixer ce critère par min- (pour min-height) ou max- (pour max-height) pour désigner respectivement la hauteur minimale et la hauteur maximale.
- device-width: signifie la largeur de l'écran. On peut préfixer ce critère par min- (pour min-device-width) ou max- (pour max-device-width) pour désigner respectivement la largeur minimale de l'écran et la largeur maximale de l'écran.
- device-height: signifie la hauteur de l'écran. On peut préfixer ce critère par min- ou max-.
- aspect-ratio: signifie le ratio (rapport entre la largeur et la hauteur) de la fenêtre d'affichage.
- device-aspect-ratio: signifie le ratio de l'écran (par exemple: 16/9 ou 4/3).
- orientation: signifie l'orientation de la zone d'affichage et peut prendre deux valeurs: portrait pour l'affichage vertical ou landscape pour l'affichage horizontale (ou paysage).

CSS : « Media Queries (Responsive Design) »

- Règle @media :
- Exemple :

```
@media all and (max-width:1280px){
    .header{
        width:100%;
        height:50px;
        text-align:center;
        background-image:url("../images/back_header_sr.png");
        border-bottom:solid 1px #E1E1E1;
        background-repeat:repeat-x;
    }
    #sfooter{
        width:100%;
        text-align:left;
        font-family:eras, "Century Gothic", verdana, sans-serif;
        color:#AAAAAA;
    }
}
@media all and (min-width:960px){
    .contdercours{
        width:30%;
        min-height:600px;
        background-color:#F4F4F4;
        float:right;
    }
    .contaccueil{
        width:68%;
        min-height:600px;
        float:left;
        border-left:dotted 1px #165877;
    }
}
```

CSS : Framework « Bootstrap »

- Bootstrap est un framework front-end open-source conçu pour faciliter et accélérer la création d'interfaces utilisateur responsives et modernes.
- **Etapes de mise en place :**
 - **Intégration :**
 - CDN : Idéal pour les prototypes ou les petits projets.
 - Téléchargement local : Plus fiable si vous travaillez hors ligne.
 - npm : Adapté pour les projets modernes utilisant des outils comme React, Vue, ou Angular.
 - **Documentation :** <https://getbootstrap.com/docs/5.3/getting-started/introduction/>

Bienvenue sur mon site

Ce site est construit avec Bootstrap

[En savoir plus](#)

150 x 150

Service 1

Description de votre premier service.

[En savoir plus](#)

150 x 150

Service 2

Description de votre deuxième service.

[En savoir plus](#)

150 x 150

Service 3

Description de votre troisième service.

[En savoir plus](#)

CSS : Framework

« Bootstrap »

```
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>Exemple Bootstrap</title>
<!-- CSS Bootstrap -->
<link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.1/dist/css/bootstrap.min.css" rel="stylesheet">
<!-- JS Bootstrap -->
<script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.1/dist/js/bootstrap.bundle.min.js"></script>
</head>
<body>
  <!-- Navigation Bar -->
  <nav class="navbar navbar-expand-lg navbar-dark bg-dark">
    <div class="container-fluid">
      <a class="navbar-brand" href="#">MonSite</a>
      <button class="navbar-toggler" type="button" data-bs-toggle="collapse" data-bs-target="#navbarNav">
        <span class="navbar-toggler-icon"></span>
      </button>
      <div class="collapse navbar-collapse" id="navbarNav">
        <ul class="navbar-nav">
          <li class="nav-item"><a class="nav-link active" href="#">Accueil</a></li>
          <li class="nav-item"><a class="nav-link" href="#">Services</a></li>
          <li class="nav-item"><a class="nav-link" href="#">Contact</a></li>
        </ul>
      </div>
    </div>
  </nav>
  <!-- Section Titre -->
  <header class="text-center py-5 bg-primary text-white">
    <h1>Bienvenue sur mon site</h1>
    <p>Ce site est construit avec Bootstrap</p>
    <button class="btn btn-light mt-3">En savoir plus</button>
  </header>
  <div class="container my-5">
    <div class="row">
      <div class="col-md-4">
        <div class="card">
          
          <div class="card-body">
            <h5 class="card-title">Service 1</h5>
            <p class="card-text">Description de votre premier service.</p>
            <a href="#" class="btn btn-primary">En savoir plus</a>
          </div>
        </div>
      </div>
    </div>
  </div>
  <!-- Footer -->
  <footer class="text-center py-3 bg-dark text-white">
    <p>© 2024 MonSite. Tous droits réservés.</p>
  </footer>
```

JavaScript : Introduction

- Javascript est un **langage de script orienté prototype** (un concept qui ressemble à l'orienté objet mais qui n'utilise pas les classes) **coté client** (exécuté sur le navigateur) qui repose sur les mêmes principes qu'un langage de programmation évolué.
- Inspiré des langages C et Java.
- JavaScript transforme une page statique (HTML/CSS) en application web interactive offrant une meilleure expérience utilisateur.
- **Interactivité** : permet de créer des **interactions** dynamiques (exemple : boutons cliquables, formulaires interactifs, modals).
- **Manipulation du DOM (Document Object Model)** : Modifie ou met à jour le contenu HTML/CSS en temps réel sans recharger la page (exemple : affichage/masquage d'éléments).
- **Gestion des événements** : Réagit aux actions des utilisateurs comme les clics, les mouvements de souris, ou la saisie clavier.
- **Animations avancées** : Crée des animations complexes non réalisables avec uniquement CSS (exemple : transitions basées sur les données).
- **Contrôle des données** : Valide et traite les formulaires avant leur soumission au serveur.
- **Communication serveur** : Envoie et reçoit des données (via AJAX ou Fetch API) sans recharger la page.
- **Programmation logique** : Implémente des conditions, boucles et calculs pour gérer des scénarios complexes.
- **Extensions fonctionnelles** : Intègre des API tierces (cartes, graphiques, etc.) et ajoute des fonctionnalités externes.

JavaScript : Introduction

- Javascript est un **langage de script orienté prototype** (un concept qui ressemble à l'orienté objet mais qui n'utilise pas les classes) **coté client** (exécuté sur le navigateur) qui repose sur les mêmes principes qu'un langage de programmation évolué.
- **Avantage:** Le code source Javascript est chargé avec la page (en même temps que HTML et CSS). A la fin du chargement, tous les codes sont présents chez le client. Par conséquent, l'exécution de Javascript est extrêmement rapide puisque tout est disponible sur place. Inversement aux langages coté serveur comme PHP, qui nécessitent à chaque fois l'interrogation du serveur, ce qui rend l'exécution plus lente.
- **Inconvénient:** Cependant, le fait que le code source javascript soit disponible sur le navigateur le rend par conséquent non confidentiel.

JavaScript : Introduction

- Intégration javascript sur la page HTML :
- Dans <head> : Pour des scripts critiques au chargement (ex : bibliothèques essentielles).
- Avant </body> : Pour la plupart des scripts (pratique et performant). Permet de s'assurer que tout le HTML est chargé avant l'exécution du script.

```
<body>
  <h1>Bonjour !</h1>
  <script type="text/javascript" src="script.js"></script>
</body>
```

```
<!DOCTYPE html>
<html>
  <head>
    <meta charset="UTF-8" />
  </head>
  <body>
    <script>
      alert("Bonjour");
    </script>
  </body>
</html>
```

- Avec defer : Pour charger des scripts en parallèle tout en respectant l'ordre **et après le HTML**.

```
<head>
  <script type="text/javascript" src="script.js" defer></script>
</head>
```

- Avec async : Pour des scripts indépendants où l'ordre d'exécution n'est pas important.

```
<head>
  <script type="text/javascript" src="script1.js" async></script>
</head>
```

- => Pour la plupart des cas modernes, **defer** est recommandé si les scripts sont placés dans la balise <head>.

JavaScript : Introduction

- **Commentaires :**
- **Commentaire de fin de ligne:** Il est inséré grâce au double slash (//). Tout le texte qui vient après est considéré comme commentaire jusqu'à la fin de la ligne.
- **Commentaire sur plusieurs lignes:** Il est inséré grâce à la séquence /* */. Tout le texte entre /* et */ est considéré comme commentaire même s'il est déclaré sur plusieurs lignes.

```
// Ce commentaire tient jusqu'à la fin de la ligne.  
/* Celui là, par contre,  
tient jusqu'à  
ce symbole: */
```

JavaScript : Notion de variable, constante et type

- **Variables :**
- Une variable sert à stocker une valeur (ou plusieurs) qui peut changer au cours de l'exécution du programme.
- La déclaration des variables en Javascript ne demande pas de préciser le type, le typage est géré dynamiquement.

```
a="Bonjour";  
var b="Salut";
```

- Pour identifier une variable on lui donne un nom unique (qui ne doit pas être attribué à une autre variable dans le même programme). Le nom de la variable peut être composé de lettres (minuscules, majuscules ou les deux), de chiffres (de 0 à 9) et les caractères _ et &. Il doit impérativement commencer par une lettre ou le caractère _.

```
maVariable=10; // Juste  
_maVariable=10; // Juste  
lmaVariable=10; // Faux  
ma Variable=10; // Faux
```

JavaScript : Notion de variable, constante et type

- **Variables :**
- **Type de variables :**
 - **Variable globale:** Cette variable est accessible de partout dans le programme, à l'intérieur des fonctions comme à l'extérieur.
 - **Variable locale:** On parle plus précisément d'une variable locale à une fonction. C'est une variable qui est accessible de l'intérieur de la fonction dans laquelle elle est déclarée, mais pas de l'extérieur.
 - **Variable locale à un bloc:** Le bloc dans ce cas peut être le corps d'une condition ou d'une boucle. Une variable dont la portée est limitée au bloc est précédée par le mot-clé **let** lorsqu'elle est initialisée dans le bloc en question.
- Si la variable est déclarée sans le mot clé var => elle est automatiquement globale qu'elle soit déclarée à l'intérieur d'une fonction ou à l'extérieur.
- Si la variable est déclarée avec le mot clé var =>
 - Si la variable est déclarée dans une fonction => elle est locale à cette fonction.
 - Si la variable est déclarée en dehors de toute fonction => elle est globale à tout le programme.

JavaScript : Notion de variable, constante et type

- **Variables :**
- **Type de variables :**
 - **Les nombres:** Ce sont les nombres avec ou sans virgule, positifs ou négatifs.
 - **Les chaînes de caractères:** (ou string), ce sont des suites de caractères quelconques.
 - **Les booléens:** Ce sont les valeurs vrai (true) ou faux (false).
 - **Les variables vides (Null):** Ce sont des variables qui ne contiennent aucune valeur.
 - **Les variables non définies (undefined):** non reconnues par le moteur Javascript.
- Au cours du même programme on peut affecter à une variable n'importe quelle valeur de n'importe quel type. Le CAST est donc fait automatiquement.

```
a=12; // Nombre  
a="Bonjour"; // Chaîne de caractères  
a=true; // Booléen
```

JavaScript : Notion de variable, constante et type

- Variables :
- Constante : Les constantes se déclarent avec le mot réservé **const** (à la place de var) et sont assignées lors de la déclaration, puis il est par définition impossible de les modifier ultérieurement.

```
const x = 24;
```

JavaScript : Notion de variable, constante et type

- **Opérateurs :**

- Les opérateurs sont des symboles qui permettent de faire des opérations (arithmétiques, logiques...) sur les variables.

- **Familles des opérateurs :**

- En Javascript (comme pour les autres langages) on distingue 5 familles d'opérateurs:

- **Opérateurs arithmétiques**: opérateurs qui effectuent des calculs mathématiques classiques à savoir: l'addition (+), la soustraction (-), la multiplication (*), la division (/) et le modulo (qui signifie le reste de la division) (%).

```
a=10;  
b=20;  
c=a+b; // c vaut 30  
d=a-b; // d vaut -10  
e=a*b; // e vaut 200  
f=a/b; // f vaut 0.5  
g=a%b; // g vaut 10
```

- **Opérateurs d'incrément**: opérateurs qui permettent de modifier la valeur d'une variable en l'augmentant ou la diminuant de 1. Deux opérateurs sont utilisés: incrément (+) et décrémentation (--).

```
a=10;  
a++; // a vaut donc 11  
a--; // a vaut à nouveau 10
```

JavaScript : Notion de variable, constante et type

▪ Opérateurs :

- Les opérateurs sont des symboles qui permettent de faire des opérations (arithmétiques, logiques...) sur les variables.

▪ Familles des opérateurs :

- En Javascript (comme pour les autres langages) on distingue 5 familles d'opérateurs:

- Opérateurs assignement: permettent d'affecter une valeur à une variable directement ou à travers une opération (comme l'incréméntation ou la décrémentation). Les opérateurs disponibles pour cette famille sont: l'affectation directe (=), l'affectation avec addition (+=), l'affectation avec soustraction (-=), l'affectation avec multiplication (*=), l'affectation avec division (/=) et l'affectation avec modulo (%=).

```
a=10;  
b=a; // b vaut 10  
a+=5; // a vaut 15  
a-=3; // a devient 12
```

- Opérateurs de comparaison: testent une condition si elle est vraie ou fausse. Les opérateurs disponibles sont: égal (==), strictement inférieur (<), inférieur ou égal (<=), strictement supérieur (>), supérieur ou égal (>=), différent (!=), identique (===), non identique (!==).

```
a=10;  
if (a%2==0)  
    res="Nombre pair";
```

JavaScript : Notion de variable, constante et type

- **Opérateurs :**
- Les opérateurs sont des symboles qui permettent de faire des opérations (arithmétiques, logiques...) sur les variables.
- **Familles des opérateurs :**
- En Javascript (comme pour les autres langages) on distingue 5 familles d'opérateurs:
 - **Opérateurs logiques**: regroupent logiquement plusieurs conditions. Les trois opérateurs utilisés sont: ET logique (&&), OU logique (||) et NON logique (!).

```
a=10;  
b=5;  
if(a%2==0 || b%2==0)  
    res="Le résultat de la multiplication de a et b sera pair";
```

JavaScript : Notion de variable, constante et type

- **Chaine de caractères (string) :**

- Une chaîne de caractères (String) est un type de variable qui correspond à une suite de caractères quelconques.

- **Création d'une chaine de caractères :**

```
str="Bonjour"; // Correcte  
str='Bonjour'; // Correcte  
str=Bonjour; // Incorrecte (*)
```

- **Concaténation des chaines** : regrouper plusieurs chaines de caractères dans une seule (soit pour l'affichage ou pour un autre traitement).

- Pour concaténer deux chaînes de caractères on utilise l'opérateur « + ».

- Supposons que nous disposons de deux variables a et b:
 - Si a et b sont tous les deux des nombres => a+b calculera la somme des deux variables.
 - Si, au moins, l'un des deux variables a ou b est une chaîne de caractère => a+b mettra les valeurs des deux variables.

```
a="Bonjour";  
b=" a tous."  
c=a+b; // Donne "Bonjour à tous."
```

```
a="Bonjour";  
b=10;  
c=a+b; // Donne "Bonjour10";
```

```
a="12";  
b=10;  
c=a+b; // Donne "1210"
```

```
a=12;  
b=10;  
c=a+b; // Donne 22
```

JavaScript : Notion de variable, constante et type

- **Chaine de caractères (string) :**
- Une chaîne de caractères (String) est un type de variable qui correspond à une suite de caractères quelconques.
- **Concaténation des chaines :**
- Si on veut vraiment calculer la somme des deux nombres a (string) et b (nombre) => La solution est la fonction **parseInt()**.
- La fonction parseInt() permet de parser (ou transformer) le type de la variable passée en paramètres en entier. Par exemple parseInt("12") retourne l'entier 12.

```
a="12";  
b=10;  
c=parseInt(a)+b; // Ce qui donne 22
```

JavaScript : Notion de variable, constante et type

- Var VS Let :

```
function testVar() {  
  if (true) {  
    var x = 10;  
  }  
  console.log(x); // Accessible here; outputs 10  
}  
testVar();
```

```
function testLet() {  
  if (true) {  
    let x = 10;  
  }  
  console.log(x); // ReferenceError: x is not defined  
}  
testLet();
```

JavaScript : Les structures conditionnelles et répétitives

- **Structure if else :**
- La structure **if else** permet de vérifier si une condition est vraie.

```
if(condition){  
    traitement 1;  
}  
else{  
    traitement 2;  
}
```

```
if(condition 1){  
    traitement 1;  
}  
else{  
    if(condition 2)  
        traitement 2;  
    else  
        traitement 3;  
}
```

```
if (condition1) {  
    traitement1;  
} else if (condition2) {  
    traitement2;  
} else {  
    traitement3;  
}
```

- **Exemple :**

```
a=10;  
if(a%2==0)  
    let b="Nombre pair";  
else  
    let b="Nombre impair";
```

JavaScript : Les structures conditionnelles et répétitives

- **Structure if else :**
- Opérateur ternaire : une écriture abrégée de la structure if else. Il est souvent utilisé si on souhaite retourner un résultat en fonction de l'état d'une condition.

```
(condition)?(traitement 1):(traitement 2);
```

- **Exemple :**

```
a=10;  
b=(a%2==0)?("Nombre pair"):("Nombre impair");
```

JavaScript : Les structures conditionnelles et répétitives

- **Structure for :**
- Permet d'exécuter le même bloc instructions plusieurs fois.
- Utilisation d'une variable compteur : a une valeur initiale, une valeur finale et un pas d'incrément.
- **Exemple :**

```
str="";  
for(i=1;i<5;i++){  
    str+=i;  
}
```

JavaScript : Les structures conditionnelles et répétitives

- **Structure while :**
- Exécute le même bloc plusieurs fois, tant que la condition est vraie.
- **Exemple :**

```
str="";  
i=1;  
while(i<5){  
    str+=i;  
    i++;  
}
```

JavaScript : Les structures conditionnelles et répétitives

Structure while : Arrêt prématuré: instruction break

- Permet d'arrêter une boucle avant d'atteindre la valeur finale du compteur.
- Cet arrêt s'applique aux boucles for et while.
- **Exemple :**

```
str="";
i=1;
while(i<5){
    if(i==3){
        break;
    }
    str+=i;
    i++;
}
```

JavaScript : Les structures conditionnelles et répétitives

Structure while : Saut d'une itération: instruction continue

- Permet de sauter l'itération courante et passer immédiatement à l'itération suivante.
- **Exemple :**

```
str="";
i=0;
while(i<5){
    i++;
    if(i==3){
        continue;
    }
    str+=i;
}
```

JavaScript : Les structures conditionnelles et répétitives

Structure switch / case

- Permet de faire plusieurs tests sur une condition.
- A la différence de la structure if / else, switch / case permet de faire moins d'imbrication de structures les unes dans les autres, ce qui simplifie le code.
- **Exemple :**

```
a=3;
switch(a) {
    case 1: b="Un"; break;
    case 2: b="Deux"; break;
    case 3: b="Trois"; break;
    default: b="Aucun des trois";
}
```

JavaScript : Les fonctions

- **Fonction** : procédure ou un sous-programme qui contient un ensemble d'instructions. Ces instructions ne seront exécutées que suite à l'appel de la fonction.
- **Function hoisting** : (ou élévation de fonctions) consiste au fait que le moteur Javascript place automatiquement au tout début du script toute fonction déclarée (même si sa déclaration est placée avant la fonction).
- **Bonne pratique de codage** :
 - Toujours déclarer les fonctions avant de les appeler.
 - Placer le JS sur la balise **head**.
- **Avantages** :
 - Réutilisation du code: via une appellation à plusieurs reprises.
 - Réduction de la taille du code: suite à sa réutilisation.
 - Organisation du code: impose une structuration du script javascript.
 - Récursivité: Une fonction peut se rappeler par elle-même.

JavaScript : Les fonctions

- **Déclaration** : commence toujours par le mot clé **function**, suivi du nom que l'on souhaite donner à la fonction (qui vérifie les mêmes règles que le nom d'une variable) suivi de parenthèses qui peuvent contenir la liste des arguments séparés par une virgule, suivi des accolades.
- Les accolades renferment le corps de la fonction qui représente l'ensemble des instructions à exécuter lors de son appel.
- La fonction peut retourner une valeur à l'aide de l'instruction **return**.

```
function Nom_de_la_fonction(liste_des_arguments){  
    Corps_de_la_fonction;  
    return Valeur_de_retour;  
}
```

test2.html

Cette page indique

Bonjour à tous!

OK

- **Exemple** : pas d'arguments et pas de retours

```
function afficher(){  
    alert("Bonjour à tous!");  
}
```

```
function afficher(){  
    alert("Bonjour à tous!");  
}  
afficher(); // Appel de la fonction afficher()
```

JavaScript : Les fonctions

- **Exemple** : arguments et pas de retours

```
function afficher(salut){  
    alert(salut+" à tous!");  
}  
afficher("Bonsoir");
```

- **Exemple** : arguments et retours

```
function afficher(salut, personne) {  
    str=salut+" à "+personne;  
    return str;  
}  
alert(afficher("Bonsoir", "vous"));
```

JavaScript : Les fonctions

- **Portée des variables :**

- Si une variable avec le mot clé **var** déclarée en dehors de la fonction aura une portée globale.
- Si une variable avec le mot clé **var** déclarée dans la fonction aura une portée locale à cette fonction.
- Si une variable sans le mot clé **var** déclarée aura une portée globale (quel que soit l'endroit de sa déclaration).

- **Exemple :**

```
a=10;
var b=20;
function f(){
    c=30;
    var d=40;
    alert(a); // Affiche 10
    alert(b); // Affiche 20
}
f();
alert(c); // Affiche 30
alert(d); // Erreur
```

JavaScript : Les fonctions

Fonction réursive

- C'est une fonction dans laquelle figure un appel à elle-même.
- Il faut toujours prévoir une **condition de sortie** qui cessera son exécution. Sinon, la fonction se rappellera indéfiniment.
- Le retour d'une valeur **return**, **return true**, **return false**, **return valeur** force l'arrêt de la fonction et par conséquent l'arrêt de la récursivité.

- Syntaxe :

```
function f(){  
    f(); // Appel récursif  
}  
f(); // Premier appel
```

- Exemple :

```
a=0;  
function f(){  
    a+=1;  
    if(a>=5)  
        return;  
    f(); // Appel récursif  
}  
f(); // Premier appel  
alert(a);
```

JavaScript : Les fonctions

Fonction réccursive

- Exercice d'application : Définir la valeur de i après l'exécution de ce code

```
i=0;  
function f() {  
    i+=1;  
}  
function g() {  
    i+=1;  
    f();  
}  
g();
```

JavaScript : Les fonctions

Fonction réursive

- Exercice d'application : Comparer ces 2 codes

```
s=0;
for(i=0;i<5;i++) {
    s+=i;
}
alert(s);
```

```
s=0;
i=0;
function f() {
    s+=i;
    i+=1;
    if(i==5)
        return;
    f();
}
f();
alert(s);
```

JavaScript : Les évènements

- Un **évènement** est une action de l'utilisateur qui peut se produire sur le navigateur, exemple : le clic.
- Le navigateur écoute les événements qui se produisent sur la page Web et lorsque arrivent les transmet au script via le gestionnaire d'événements.
- Le **gestionnaire d'événements** permet d'associer un traitement à un événement.
- **Syntaxe** : **onÉvènement = "code Javascript ou appellation fonction"**
- **Exemple :**

```
<!DOCTYPE html>
<html>
  <head>
    <meta charset="UTF-8" />
    <script language="javascript">
      function f(){
        alert("Vous avez bien cliqué sur le bouton!");
      }
    </script>
  </head>
  <body>
    <input type="button" value="Cliquez ici" onClick="f()" />
  </body>
</html>
```

JavaScript : Les évènements

Liste des évènements javascript les plus utilisés

- **click** (onClick) : capturé sur un objet quand on clique dessus. Utilisé sur les boutons, images, hyperliens, vidéos...
- **dblClick** (onDbClick) : clique sur un objet deux fois de suite (double clic).
- **mouseover** (onMouseOver) : Se déclenche lorsqu'un pointeur de souris entre dans la zone d'un élément.
- **mouseout** (onMouseOut) : Se déclenche lorsqu'un pointeur de souris quitte la zone d'un élément HTML.
- **focus** (onFocus) : lorsque le curseur est placé dans un champ du formulaire, par click ou par tabulation.
- **blur** (onBlur) : lorsqu'un élément perd le focus.
- **keyDown** (onKeyDown) : lorsqu'une touche du clavier est enfoncée.
- **keyUp** (onKeyUp) : quand une touche du clavier est relâchée.
- **keyPress** (onKeyPress) : quand une touche du clavier est maintenue enfoncée.
- **load** (onLoad) : quand un élément est chargé par le navigateur. Elle peut être appliquée à la page entière (balise <body>), dans ce cas l'événement se produira quand tous les éléments de la page seront chargés.

JavaScript : Les évènements

Liste des évènements javascript les plus utilisés

- **resize** (onResize) : au redimensionnement de la taille de la fenêtre du navigateur.
- **select** (onSelect) : lors de la sélection du texte contenu dans l'objet accueillant l'évènement.
- **change** (onChange) : quand l'internaute change le contenu d'un élément (liste de sélection ou zone de texte par exemple).
- **submit** (onSubmit) : au clic sur n'importe quel bouton de type submit présent dans la page (ou dans le formulaire).
- **mousedown** (onMouseDown) : quand l'internaute appuie sur n'importe quel bouton de la souris.
- **mouseup** (onMouseUP) : quand le bouton de la souris est relâché.
- **mousemove** (onMouseMove) : Quand l'internaute fait bouger le curseur de la souris dans la zone accueillant l'évènement. Cet évènement est différent du hover (entrer et sortir de l'élément).
- **dragStart** (onDragStart) : au commencement du déplacement d'un élément par "glisser-déposer" (Drag & Drop).
- **dragEnter** (onDragEnter) : Se déclenche une seule fois lorsque l'élément est entré par un élément en cours de déplacement (drag).
- **dragOver** (onDragOver) : Se déclenche continuellement tant qu'un élément en cours de déplacement survole la zone cible.
- **drop** (onDrop) : au dépôt de l'élément glissé dans la zone prévue à cet effet.

JavaScript : Les évènements

Liste des évènements javascript les plus utilisés

- **resize** (onResize) : au redimensionnement de la taille de la fenêtre du navigateur.
- **select** (onSelect) : lors de la sélection du texte contenu dans l'objet accueillant l'évènement.
- **change** (onChange) : quand l'internaute change le contenu d'un élément (liste de sélection ou zone de texte par exemple).
- **submit** (onSubmit) : au clic sur n'importe quel bouton de type submit présent dans la page (ou dans le formulaire).
- **mouseDown** (onMouseDown) : quand l'internaute appuie sur n'importe quel bouton de la souris.
- **mouseUp** (onMouseUP) : quand le bouton de la souris est relâché.
- **mousemove** (onMouseMove) : Quand l'internaute fait bouger le curseur de la souris dans la zone accueillant l'évènement. Cet évènement est différent du hover (entrer et sortir de l'élément).
- **dragStart** (onDragStart) : au commencement du déplacement d'un élément par "glisser-déposer" (Drag & Drop).
- **dragEnter** (onDragEnter) : Se déclenche une seule fois lorsque l'élément est entré par un élément en cours de déplacement (drag).
- **dragOver** (onDragOver) : Se déclenche continuellement tant qu'un élément en cours de déplacement survole la zone cible.
- **drop** (onDrop) : au dépôt de l'élément glissé dans la zone prévue à cet effet.

JavaScript : Les évènements

- Syntaxe :

- Façon 1 : addEventListener

```
element.addEventListener('nom_evenement', fonction_a_executer);
```

```
element.addEventListener('dragenter', (event) => {  
    console.log('Événement dragenter détecté.');
```

```
});
```

- Façon 2 : La propriété de l'élément DOM

```
element.onnom_evenement = fonction_a_executer;
```

```
element.ondragenter = () => {  
    console.log('Événement dragenter détecté.');
```

```
};
```

- Façon 3 : un attribut dans le HTML

```
<element onnom_evenement="fonction_a_executer()"></element>
```

```
<div ondragenter="handleDragEnter()">Faites glisser ici</div>
```

```
<script>
```

```
    function handleDragEnter() {  
        console.log('Événement dragenter détecté.');
```

```
    }
```

JavaScript : Le Document Object Model (DOM)

- Javascript est un langage de programmation orienté prototype. Il s'agit d'un concept proche de celui de la programmation orienté objet (POO), mais qui n'utilise pas les classes.
- Javascript est considéré comme une extension du langage HTML => la plupart de ses objets (prototypes) sont en réalité des objets HTML présents dans le document (comme les images, les champs de formulaire...)
- Il existe aussi des objets propres au Javascript qui permettent d'exécuter des traitements comme le calcul, la manipulation de la date, le traitement des chaînes de caractères...
- Les objets Javascript qui dépendent du contenu d'un document HTML sont structurés hiérarchiquement. C'est à dire que des objets s'emboîtent dans d'autres jusqu'à atteindre l'objet le plus élevé à l'échelle de la hiérarchie.
- L'objet le plus élevé est la fenêtre du navigateur, il est identifié par le nom window. Cet objet peut être manipulé directement à travers des fonctions qui lui sont prédéfinies. On appelle ces fonctions des méthodes. A titre d'exemple `alert()` est une méthode de l'objet window, on aurait pu donc l'écrire comme ceci: `window.alert()`.
- Le point (.) permet de passer d'un objet à une de ces méthodes ou d'un objet parent à un autre objet enfant, ou encore d'un objet à une variable qui lui est propre. On appelle cette variable un attribut.

JavaScript : Le Document Object Model (DOM)

- Un objet peut représenter un élément HTML (image, texte, formulaire...) ou un élément propre au Javascript qui sert à exécuter des traitements spéciaux.
- Une méthode est une fonction prédéfinie appartenant à l'objet et qui permet de faire les traitements sur celui-ci.
- Un attribut est une variable qui appartient à l'objet. Elle représente une propriété ou caractéristique de celui-ci. On peut le manipuler grâce aux méthodes ou directement.
- L'objet **window** contient un sous-objet, c'est le document HTML. Cet objet est identifié par le nom document. Cet objet peut contenir les éléments HTML conventionnels comme du texte, les images, les formulaires... L'objet formulaire par exemple contient à son tour des sous-objets, ce sont les champs de formulaire etc...
- Le navigateur est inclus dans sa fenêtre du nom de **window**, c'est l'objet en tête de la hiérarchie. Puis il y a le document HTML du nom de document qui vient après. Donc le nom complet de l'objet document est **window.document**.
- Cependant, la déclaration de l'objet window n'est pas obligatoire. Exemple, l'appel de la méthode alert() sans le préfixe window.

JavaScript : Le Document Object Model (DOM)

- Exemple d'application :
- L'objet document contient un formulaire dont l'attribut name vaut fo. Alors, ce formulaire est identifié en Javascript par son nom fo ou plutôt par son nom complet **window.document.fo**.
- Le formulaire fo renferme deux champs de formulaire. Le Javascript identifiera l'objet zone de texte du nom de login par son nom **window.document.fo.login** ou tout juste **document.fo.login**.

```
<!DOCTYPE html>
<html>
  <head>
    <meta charset="UTF-8" />
  </head>
  <body>
    <form name="fo">
      <input type="text" name="login" />
      <input type="button" value="Action" />
    </form>
  </body>
</html>
```

JavaScript : Le Document Object Model (DOM)

- **DOM** (Document objet Model) est un outil qui permet d'accéder au document. Il s'agit plus précisément d'une interface qui standardise la manière d'accès aux documents structurés en balises comme HTML ou XML. Chaque élément du document fait partie du modèle DOM.
- **Accès aux objets :**
- **getElementById():** permet d'accéder à un objet quelconque en l'identifiant par la valeur de son attribut id.
- **getElementsByTagName():** permet d'accéder tous les objets correspondant à la balise (tag) déclarée entre les parenthèses en tant que chaîne de caractères. Elle retourne un tableau qui renferme tous les nœuds du type désigné.
- **getElementsByClassName():** permet d'accéder à un objet quelconque en l'identifiant par la valeur de son attribut id.

```



<script language="javascript">
    document.getElementById("im1").src="image4.jpg";
    for(i=1;i<document.getElementsByTagName("img").length;i++)
        document.getElementsByTagName("img")[i].src="image4.jpg";
</script>
```

JavaScript : Le Document Object Model (DOM)

- **Accès aux objets :**
- **Attribut innerHTML** : permet d'écrire du contenu dans la balise <body>

```
<div id="contenu">Bonjour</div>
<script language="javascript">
  document.getElementById("contenu").innerHTML="Bonsoir";
</script>
```

```
<html lang="fr">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Exemple innerHTML</title>
</head>
<body>
  <div id="conteneur"></div>

  <script>
    // Sélection de la div
    const conteneur = document.getElementById('conteneur');

    // Contenu HTML à insérer
    const contenu = `
      <h1>Bienvenue</h1>
      <p>Ceci est un paragraphe généré avec innerHTML.</p>
      <ul>
        <li>Élément 1</li>
        <li>Élément 2</li>
        <li>Élément 3</li>
      </ul>
    `;

    // Insertion dans la div
    conteneur.innerHTML = contenu;
  </script>
</body>
```

JavaScript : Le Document Object Model (DOM)

- **Accès aux objets :**
- **Attribut className** : renomme l'attribut class d'un élément. C'est pratique pour appliquer des styles CSS dynamiquement à l'objet en question.

```
<!DOCTYPE html>
<html>
  <head>
    <meta charset="UTF-8" />
    <style>
      .nouvelleClasse{
        color:#FFFFFF;
        font-size:18pt;
        background-color:#000000;
        padding:10px;
      }
    </style>
    <script language="javascript">
      function changer(){
        document.getElementsByTagName("h1").item(0).className="
nouvelleClasse";
      }
    </script>
  </head>
  <body>
    <h1>Titre d'order 1</h1>
    <input type="button" value="Changer l'apparence"
onclick="changer()" />
  </body>
</html>
```

JavaScript : Le Document Object Model (DOM)

- **Accès aux objets :**
- **Attribut style :** Le sous-objet style permet de définir dynamiquement un attribut de style sur la balise concernée.

```
<span id="citation">
  Mieux vaut tard que jamais.
</span>
<script language="javascript">
  document.getElementById("citation").style.color="#888888";
  document.getElementById("citation").style.fontStyle="italic";
</script>
```

JavaScript : Le Document Object Model (DOM)

- **Accès aux objets :**
- **Méthode `setAttribute()`** : permet de changer dynamiquement la valeur d'un attribut d'une balise alors on fait appel à l'attribut `setAttribute`.

```
<div align="left">
    Contenu de la DIV.
</div>
<script language="javascript">
    document.getElementsByTagName("div").item(0).setAttribute("align", "center");
</script>
```

```
// Au lieu de
document.getElementsByTagName("h1").item(0).className="nouvelleClasse";

// On peut mettre
document.getElementsByTagName("h1").item(0).setAttribute("class", "NouvelleClasse");
```

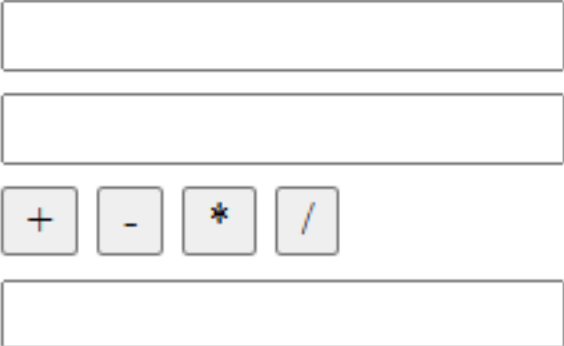
JavaScript : Le Document Object Model (DOM)

- **Accès aux objets :**
- **Méthode `getAttribute()`** : permet de récupérer la valeur de l'attribut passé en paramètre.

```
<video src="pub.ogv" id="publicite"></video>
<script language="javascript">
    alert(document.getElementById("publicite").getAttribute("src"));
</script>
```

JavaScript :

- **Exercice d'application**
- Mettez en place la calculatrice suivante :



A simple calculator interface is shown within a blue-bordered box. It consists of two input fields for numbers, a row of four operator buttons (+, -, *, /), and a final input field for the result.

JavaScript : Les tableaux

- Les tableaux sont un type spécial de variables qui permettent de contenir plusieurs valeurs à la fois.
- Un tableau est un objet qui peut contenir des entrées du même type ou de types différents.

- **Syntaxes :**

- **Syntaxe 1 :**

```
tab=new Array();  
tab[0]="Javascript";  
tab[1]="Coté client";  
tab[2]=40;
```

- **Syntaxe 2 :**

```
tab=["Javascript","Coté client",40];
```

- **Syntaxe 3 :**

```
tab=new Array("Javascript","Coté client",40);
```

JavaScript : Les tableaux

- **Attributs et méthodes :**
- **length**: retourne la taille du tableau.
- **sort()**: retourne le tableau trié dans l'ordre alphanumérique croissant.
- **reverse()**: retourne le tableau dans l'ordre inversé des éléments.
- **join()**: retourne une chaîne de caractères constituée des éléments du tableau séparés par l'occurrence passée en paramètres (entre les parenthèses).

```
tab=new Array("A","B",10);
tab.length;
/* Retourne 3 */

tab.sort();
/* Retourne un tableau dont les éléments sont déclarés dans cet
ordre: 10, A, B */

tab.reverse();
/* Retourne un tableau dont les éléments sont déclarés dans cet
ordre: 10, B, A */

tab.join("-");
/* Retourne la chaîne de caractères "A-B-10" */

tab.sort().reverse().join("-");
/* Retourne la chaîne de caractères: "B-A-10" */
```

JavaScript : Les tableaux

- **Parcours du tableau :**

- **Méthode 1 : for**

```
str="";
for(i=0;i<3;i++){
    for(j=0;j<3;j++){
        str+=tab[i][j]+" ";
    }
    str+="n";
}
alert(str);
```

- **Méthode 2 : ForEach**

- un itérable Javascript qui permet de parcourir un tableau élément par élément tout en effectuant un traitement. Elle accepte comme argument une fonction qui à son tour accepte comme argument une variable qui vaudra l'élément courant du tableau au moment de son parcours.

```
tab=["Javascript","Coté client",40];
tab.forEach(function(x){console.log(x)});
```

```
tab=["Javascript","Coté client",40];
tab.forEach(x=>console.log(x));
```

JavaScript : Les chaines de caractères

- Une chaîne de caractère représente un objet javascript : l'objet String.

- **Attributs et méthodes :**

```
str="Bonjour";
```

- **length**: retourne le nombre de caractères contenus dans la chaîne.
- **charAt(x)**: retourne le caractère qui se trouve à la position **x** passé en paramètre. La valeur 0 indexe le premier caractère de la chaîne.
- **indexOf(car)**: retourne la position du caractère **car** passé en paramètre. Si le caractère existe dans la chaîne, alors sa position (comprise entre 0 et la longueur de la chaîne - 1) est retournée, sinon (le caractère ne figure pas dans la chaîne) alors la valeur -1 est retournée. La méthode **indexOf()** peut accueillir un deuxième paramètre qui est un entier qui indique à partir de quelle position de la chaîne on commencera la recherche du caractère passé en premier paramètre.
- **substring(debut,fin)**: extrait une partie de la chaîne de caractères commençant de la position **debut** et finissant à la position **fin-1**.
- **split(occurence)**: retourne un tableau à partir des fractions de la chaîne de caractères obtenues en divisant celle-ci au niveau de l'occurence.
- **toLowerCase()**: retourne la chaîne de caractère entièrement en minuscules.
- **toUpperCase()**: retourne la chaîne de caractère entièrement en majuscules.

JavaScript : Les chaines de caractères

- Exemple :

```
str="Bonjour";
str.length;
/* Retourne 7 */

str.charAt(0);
/* Retourne B */

str.indexOf("o");
/* Retourne 1 */

str.indexOf("o",2);
/* Retourne 4 */

str.indexOf("x");
/* Retourne -1 */

str.substring(0,3);
/* Retourne Bon */

str.substring(3,6);
/* Retourne jou */

tab=str.split("o");

tab.length;
/*Retourne 3 */

tab.join("-");
/*Retourne B-nj-ur */
```

JavaScript : Bibliothèques et Framework JS (AngularJS vs ReactJS)

- **JavaScript** : un langage fondamental pour le développement web.
- **ReactJS** : une bibliothèque focalisée sur la création d'interfaces utilisateur modernes.
- **AngularJS** : un framework complet idéal pour des projets structurés et de grande échelle.

Caractéristique	JavaScript	ReactJS	AngularJS
Type de technologie	Langage de programmation	Bibliothèque pour les interfaces UI	Framework complet pour les applications SPA
Créé par	Brendan Eich (Netscape) en 1995	Facebook en 2013	Google en 2010
Architecture	Non structuré	Unidirectionnelle (One-Way Data Binding)	Bidirectionnelle (Two-Way Data Binding)
Langage utilisé	Pur JavaScript	JavaScript ES6+, JSX	JavaScript et TypeScript
Concept principal	Langage général pour le développement web	Composants pour la construction de vues	MVC (Model-View-Controller)
Performance	Dépend des pratiques du développeur	Élevée grâce au Virtual DOM	Moins efficace pour les grandes applications
Courbe d'apprentissage	Facile (concepts de base simples)	Modérée (besoin de comprendre JSX et ES6)	Complexe (nombreux concepts intégrés)
Utilisation principale	Développement web de base	Interfaces utilisateur interactives	Applications SPA riches
Modèle DOM	DOM réel	Virtual DOM	DOM réel
Support des outils	Utilisé avec des outils comme Node.js, jQuery	Besoin de bibliothèques complémentaires	Outils intégrés (routage, injection, etc.)
Flexibilité	Très flexible mais nécessite des bibliothèques	Très flexible avec des bibliothèques	Moins flexible mais plus structuré
Taille du projet	Petits à grands	Petits à moyens (ou grands avec Redux)	Grands projets d'entreprise
Exemple d'utilisation	Ajout de scripts interactifs à une page HTML	Interfaces interactives comme Instagram	Applications complexes comme Gmail
Popularité et support	Très populaire, base de tout développement web	Très populaire pour UI modernes	Solide mais moins populaire que React

Guidelines pour l'examen

- **Contenu :**
 - Portera sur toutes les parties du **Cours** (sauf Bootstrap / AngularJS / ReactJS)
 - Portera sur tous les **TPs de 1 à 10**. Les parties ne sont pas comprises.
- **Réponses :**
 - Sur la feuille de l'examen, il faut **noircir (c'est-à-dire colorier)** la case de la réponse ou les cases des réponses. Il ne suffit pas de cocher, ni de mettre une croix.